

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Nguyễn Quang Ninh

**NGHIÊN CỨU, ĐÁNH GIÁ HIỆU NĂNG SLLMS
TRONG BÀI TOÁN SINH CÂU HỎI TRẮC NGHIỆM
TỪ ẢNH VÀ XÂY DỰNG ỨNG DỤNG THỰC NGHIỆM**

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Công nghệ thông tin

HÀ NỘI - 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



NGUYỄN QUANG NINH

NGHIÊN CỨU, ĐÁNH GIÁ HIỆU NĂNG SLLMS
TRONG BÀI TOÁN SINH CÂU HỎI TRẮC NGHIỆM
TỪ ẢNH VÀ XÂY DỰNG ỨNG DỤNG THỰC NGHIỆM

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Công nghệ thông tin

Cán bộ hướng dẫn: TS. Nguyễn Văn Sơn

HÀ NỘI - 2026

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



Nguyen Quang Ninh

**RESEARCH AND EVALUATION OF SLLMS PERFORMANCE IN
IMAGE-BASED MULTIPLE-CHOICE QUESTION GENERATION
AND EXPERIMENTAL APPLICATION DEVELOPMENT**

BACHELOR'S THESIS

Major: Information Technology

Supervisor: Dr. Nguyen Van Sơn

HANOI - 2026

Lời cảm ơn

Đầu tiên, em muốn thể hiện lòng biết ơn sâu sắc đến thầy Nguyễn Văn Sơn - người đã không ngừng hỗ trợ, đưa ra ý kiến đóng góp và chia sẻ kiến thức quý báu trong suốt quá trình nghiên cứu và thực hiện khóa luận này. Sự tận tâm và am hiểu của thầy đã giúp em vượt qua những khó khăn, giúp em hiểu biết sâu hơn về đề tài và hoàn thành các công việc liên quan một cách đúng đắn. Những ý kiến đóng góp từ thầy đã là nguồn động viên lớn lao, giúp em tiến bộ và phát triển không chỉ trong quá trình hoàn thành khóa luận mà còn trong sự nghiệp tương lai sau này.

Bên cạnh đó, em muốn bày tỏ lòng biết ơn sâu sắc tới Khoa Công nghệ Thông tin, Trường Đại học Công nghệ - Đại học Quốc gia Hà Nội, nơi đã cung cấp cho em môi trường học tập và nghiên cứu lý tưởng nhất. Những điều kiện và cơ sở vật chất tốt nhất tại đây đã tạo điều kiện thuận lợi cho quá trình học tập và phát triển bản thân. Những kiến thức và kỹ năng mà em đã học được từ các giảng viên và bạn bè sẽ là nền tảng vững chắc cho sự nghiệp và sự phát triển của em trong tương lai.

Đặc biệt, em muốn bày tỏ lòng biết ơn sâu sắc đến bố mẹ, những người đã luôn ở bên cạnh và hỗ trợ em trong suốt hành trình học tập và nghiên cứu. Sự quan tâm, động viên và sự hy sinh của họ là nguồn động viên lớn lao, giúp em vượt qua mọi khó khăn và đạt được những thành công nhỏ lẻ trong cuộc sống.

Cuối cùng, em muốn gửi lời tri ân đến tất cả những người bạn, đồng nghiệp và gia đình đã đóng góp và chia sẻ kiến thức, ý kiến và ý tưởng quý báu trong quá trình hoàn thành khóa luận này. Sự ủng hộ và sự khích lệ từ mọi người đã là nguồn động viên quan trọng, giúp em vượt qua mọi thách thức và hoàn thành công việc một cách tốt nhất.

Xin chân thành cảm ơn!

Lời cam đoan

Tôi là Nguyễn Quang Ninh, sinh viên lớp QH-2022-I/CQ-I-IT15 khóa 67 theo học ngành Công nghệ thông tin tại trường Đại học Công Nghệ - Đại học Quốc gia Hà Nội. Tôi xin cam đoan khóa luận “Nghiên cứu, đánh giá hiệu năng sLLMs trong bài toán sinh câu hỏi trắc nghiệm từ ảnh và xây dựng ứng dụng thực nghiệm” là công trình nghiên cứu do bản thân tôi thực hiện. Các nội dung nghiên cứu, kết quả trong báo cáo là xác thực.

Các thông tin sử dụng trong báo cáo là có cơ sở và không có nội dung nào sao chép từ các tài liệu mà không ghi rõ trích dẫn tham khảo. Tôi xin chịu trách nhiệm về lời cam đoan này.

Hà Nội, ngày XX tháng XX năm 2026

Sinh viên

Nguyễn Quang Ninh

Tóm tắt

Việc tự động sinh câu hỏi trắc nghiệm từ tài liệu giáo dục là nhu cầu cấp thiết trong bối cảnh số hóa giáo dục, nhưng các giải pháp hiện tại phụ thuộc vào mô hình ngôn ngữ lớn thương mại với chi phí cao và yêu cầu kết nối internet ổn định. Khóa luận này đề xuất kiến trúc đa tác tử, sử dụng các mô hình ngôn ngữ lớn cỡ nhỏ (sLLM - Small Large Language Model) để sinh câu hỏi trắc nghiệm chất lượng cao từ ảnh chụp văn bản. Hệ thống triển khai ba tác tử *sinh câu hỏi* hoạt động song song theo từng cấp độ Bloom, kết hợp với tác tử *Phân loại* để kiểm chứng cấp độ nhận thức, tác tử *Học sinh* để kiểm tra khả năng giải được và tác tử *Hiệu chỉnh* để sửa các phương án trả lời chưa đạt yêu cầu. Khung đánh giá ba tiêu chí gồm khả năng giải được dựa trên ngữ cảnh, chất lượng phương án nhiễu và độ bám sát phân loại nhận thức, kết hợp với phương pháp sử dụng LLM để đánh giá (LLM-as-a-Judge), được xây dựng để đánh giá tự động chất lượng câu hỏi. Ngoài ra, quy trình sinh dữ liệu OCR sử dụng kỹ thuật tạo ảnh tổng hợp bằng trình duyệt nhằm phát triển bộ dữ liệu đánh giá. Kết quả thực nghiệm cho thấy kiến trúc đa tác tử cải thiện chất lượng sLLM rõ rệt, với mức tăng tương đối 93,6% ở chỉ số chất lượng phương án nhiễu của Phi-4 và chi phí dưới \$0.50 cho mỗi 100 đoạn văn. Xét theo tỷ lệ chấp nhận (Acceptance), cấu hình Phi-4 kết hợp đa tác tử đạt trung bình 0,8557 trên ba cấp độ, trong đó đạt 0,9753 ở Cấp 1 và 0,9072 ở Cấp 2, cạnh tranh sát sao với Gemini 3 Flash ở hai cấp độ này; tuy nhiên, nhóm LLM thương mại vẫn duy trì ưu thế ở Cấp 3. Ứng dụng Openlet được phát triển để minh họa khả năng ứng dụng thực tế của hệ thống.

Từ khóa: Mô hình ngôn ngữ lớn cỡ nhỏ, Hệ thống đa tác tử, Câu hỏi trắc nghiệm, Nhận dạng ký tự quang học, Phân loại Bloom, LLM-as-a-Judge

Abstract

Automatic multiple-choice question (MCQ) generation from educational documents is an essential yet challenging task, particularly when relying on proprietary large language models (LLMs) that incur high costs and require persistent internet connectivity. This thesis proposes a multi-agent architecture to leverage small large language models (sLLMs) for high-quality MCQ generation from document images. The system deploys three parallel question-generation agents for different Bloom levels, together with a Classifier Agent for cognitive-level verification, a Student Agent for solvability checking, and a Fixer Agent for repairing answer options that fail quality control. A three-criteria evaluation framework (Solvability, Distractor Quality, Alignment) combined with the LLM-as-a-Judge methodology is developed for automated quality assessment. Additionally, a synthetic OCR data generation pipeline using Browser Engineering and Augraphy is introduced to create diverse evaluation datasets. Experimental results demonstrate that the multi-agent architecture substantially improves sLLM quality, increasing Phi-4's Distractor Quality by 93.6% in relative terms while keeping the cost below \$0.50 per 100 passages. In terms of Acceptance Rate, Phi-4 with multi-agent orchestration achieves an average score of 0.8557 across three Bloom levels, including 0.9753 at Level 1 and 0.9072 at Level 2, which is closely competitive with Gemini 3 Flash at these two levels; however, proprietary LLMs still retain an advantage at Level 3. The Openlet web application is developed as a full-stack demonstration integrating the complete research architecture.

Keywords: Small Large Language Model, Multi-Agent System, Multiple-Choice Question, Optical Character Recognition, Bloom's Taxonomy, LLM-as-a-Judge

Mục lục

Lời cảm ơn

Lời cam đoan i

Tóm tắt ii

Abstract iii

Mục lục iv

Danh sách hình vẽ vii

Danh sách bảng ix

Danh mục các từ viết tắt x

Chương 1 Mở đầu 1

1.1 Đặt vấn đề 1

1.2 Giải pháp đề xuất 2

1.3 Đóng góp của khóa luận 3

1.4 Cấu trúc khóa luận 4

Chương 2 Cơ sở lý thuyết & Các công trình liên quan 6

2.1 Phân loại Bloom 6

2.1.1 Cấp độ 1 - Truy xuất 6

2.1.2 Cấp độ 2 - Suy luận và tổng hợp 7

2.1.3 Cấp độ 3 - Lập luận phản biện 7

2.2 Hệ thống đa tác tử 8

2.2.1 Khái niệm hệ thống đa tác tử 8

2.2.2 Thư viện LangGraph 9

2.3 Phương pháp đánh giá bằng LLM 10

2.4	Các công trình liên quan	11
2.4.1	Sinh câu hỏi trắc nghiệm tự động bằng quy tắc	11
2.4.2	Sinh câu hỏi trắc nghiệm dựa trên mô hình ngôn ngữ lớn	12
2.5	Phân tích khoảng trống nghiên cứu	12
Chương 3 Phương pháp sinh câu hỏi trắc nghiệm bằng luồng đa tác tử		13
3.1	Kiến trúc hệ thống	13
3.2	Luồng xử lý đa tác tử	14
3.2.1	Tác tử <i>Sinh câu hỏi</i>	14
3.2.2	Tác tử <i>Phân loại</i>	16
3.2.3	Tác tử <i>Học sinh</i>	17
3.2.4	Tác tử <i>Hiệu chỉnh</i>	18
3.3	Một số kỹ thuật cải thiện chất lượng sLLM	19
3.3.1	Suy luận chuỗi tư duy	19
3.3.2	Định dạng đầu ra của tác tử	20
Chương 4 Phương pháp đánh giá		22
4.1	Bộ dữ liệu	22
4.2	Quy trình tổng hợp bộ dữ liệu OCR	23
4.3	Thiết lập thực nghiệm	25
4.4	Phương pháp và Tiêu chí Đánh giá	26
4.4.1	Đánh giá chất lượng câu hỏi trắc nghiệm	26
4.4.2	Đánh giá hiệu năng nhận dạng văn bản	28
Chương 5 Kết quả		30
5.1	Kết quả sinh câu hỏi	30
5.1.1	Kết quả nhóm sLLM cơ sở	30
5.1.2	Kết quả nhóm sLLM kết hợp đa tác tử	31
5.1.3	Kết quả nhóm LLM thương mại	32
5.1.4	So sánh tổng hợp	32
5.2	Kết quả OCR	33
5.2.1	Kết quả trên ảnh sạch	34
5.2.2	Kết quả trên ảnh nhiễu	34
5.3	Phân tích chi phí	35

5.4 Thảo luận	37
Chương 6 Thiết kế và hiện thực ứng dụng Openlet	40
6.1 Giới thiệu ứng dụng và công nghệ sử dụng	40
6.2 Phân tích đặc tả yêu cầu	41
6.2.1 Yêu cầu chức năng	41
6.2.2 Yêu cầu phi chức năng	42
6.3 Thiết kế hệ thống	44
6.3.1 Kiến trúc tổng thể	44
6.3.2 Mô hình dữ liệu	45
6.4 Hiện thực các luồng chức năng cốt lõi	47
6.4.1 Luồng tạo bài kiểm tra tự động	47
6.4.2 Luồng truy cập, làm bài và nộp bài	48
6.5 Giao diện tiêu biểu của ứng dụng	48
6.5.1 Giao diện dành cho giáo viên	48
6.5.2 Giao diện dành cho người làm bài	49
Chương 7 Kết luận và Hướng phát triển	51
7.1 Tổng kết	51
7.2 Hạn chế	51
7.3 Hướng phát triển	52
Tài liệu tham khảo	53

Danh sách hình vẽ

3.1	Kiến trúc hệ thống đa tác tử với hai vòng lặp kiểm soát chất lượng: vòng lặp phân loại cấp độ nhận thức và vòng lặp kiểm định tính giải được của câu hỏi	13
3.2	Luồng xử lý bên trong Tác tử Sinh câu hỏi: nhận văn bản nguồn và mẫu tham chiếu, sinh câu hỏi qua sLLM với prompt chuyên biệt theo cấp độ	16
3.3	Luồng xử lý bên trong Tác tử Phân loại: xáo trộn câu hỏi chống thiên lệch, phân loại cấp độ Bloom, và tách kết quả thành tập đạt chuẩn và tập sinh lại	17
3.4	Luồng xử lý bên trong Tác tử Học sinh: giải MCQ dạng nhiều lựa chọn đúng, phân luồng câu hỏi đạt chuẩn và câu hỏi cần hiệu chỉnh kèm lý do	18
3.5	Luồng xử lý bên trong Tác tử Hiệu chỉnh: nhận câu hỏi lỗi kèm lý do từ Tác tử Học sinh, chỉ sửa phương án và đáp án, đưa lại để tái kiểm định	19
3.6	Cấu trúc định dạng đầu ra chuẩn của tác tử <i>Sinh câu hỏi</i>	21
4.1	Sơ đồ quy trình tăng cường dữ liệu ảnh nhiễu: từ văn bản gốc qua kết xuất ảnh sạch, áp dụng nhiễu vật lý và quang học, đến ảnh giả lập tài liệu thực tế	23
4.2	Các mẫu ảnh tài liệu tổng hợp minh họa các loại nhiễu được áp dụng: nhiễu vật liệu (vết ố vàng, nhòe mực), nhiễu quang học (bóng đổ, nhiễu hạt), và biến dạng hình học (cong vênh trang)	24
5.1	So sánh chỉ số Acceptance trung bình giữa ba nhóm mô hình (sLLM cơ sở, sLLM kết hợp MAS, LLM thương mại) theo ba cấp độ nhận thức Bloom	33

5.2	So sánh CER và WER giữa điều kiện ảnh sạch và ảnh nhiễu cho bảy mô hình OCR, minh họa mức độ suy giảm hiệu suất khi chuyển sang điều kiện thực tế	36
5.3	Biểu đồ phân tán chi phí API (USD) so với Acceptance trung bình trên ba cấp độ nhận thức, xác định biên Pareto tối ưu giữa chất lượng và ngân sách	37
6.1	Biểu đồ ca sử dụng tổng quát của Openlet với hai tác nhân chính: giáo viên (quản lý vòng đời bài kiểm tra) và người làm bài (truy cập và nộp bài)	43
6.2	Kiến trúc ba tầng của Openlet: tầng giao diện (Next.js/React), tầng dịch vụ (Firebase), và tầng AI (OpenRouter kết nối các mô hình OCR/LLM)	44
6.3	Mô hình dữ liệu chính của Openlet gồm ba bảng: users (hồ sơ người dùng), quizzes (bài kiểm tra và câu hỏi), và attempts (lượt làm bài và kết quả)	46
6.4	Biểu đồ tuần tự của luồng tạo bài kiểm tra tự động: từ tiếp nhận tài liệu đầu vào, qua OCR và MAS sinh câu hỏi, đến cập nhật trạng thái theo thời gian thực	47
6.5	Biểu đồ tuần tự của luồng truy cập, làm bài và nộp bài: người làm bài nhận đề đã lọc đáp án, chấm điểm phía máy chủ, và nhận phản hồi theo mức cấu hình	48
6.6	Giao diện biên tập bài kiểm tra dành cho giáo viên: hiển thị câu hỏi đã sinh tự động với các công cụ chỉnh sửa nội dung, phương án và cấp độ nhận thức	49
6.7	Giao diện làm bài (trái) và trang kết quả (phải): người làm bài chọn đáp án với bộ đếm thời gian, sau đó nhận phản hồi theo mức hiển thị cấu hình bởi giáo viên	50

Danh sách bảng

4.1	Thống kê bộ dữ liệu RACE và mẫu thực nghiệm	23
4.2	Danh sách 12 mô hình sử dụng trong thực nghiệm	25
5.1	Kết quả đánh giá nhóm sLLM cơ sở trên bộ dữ liệu RACE-High . .	30
5.2	Kết quả đánh giá nhóm sLLM đa tác tử trên bộ dữ liệu RACE-High	31
5.3	Kết quả đánh giá nhóm LLM thương mại trên bộ dữ liệu RACE-High	32
5.4	Kết quả OCR trên ảnh sạch	34
5.5	Kết quả OCR trên ảnh nhiễu	35
5.6	Chi phí sử dụng API cho 100 đoạn văn mẫu (USD)	36
6.1	Các Cloud Functions cốt lõi của Openlet và vai trò trong hệ thống .	45

Danh mục các từ viết tắt

Từ viết tắt	Cụm từ đầy đủ	Ý nghĩa / Mô tả
AMCQG	Automated Multiple-Choice Question Generation	Sinh câu hỏi trắc nghiệm tự động
API	Application Programming Interface	Giao diện lập trình ứng dụng
CAT	Computerized Adaptive Testing	Kiểm tra thích ứng trên máy tính
CER	Character Error Rate	Tỷ lệ lỗi ký tự
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
CoT	Chain-of-Thought	Chuỗi suy luận
LLM	Large Language Model	Mô hình ngôn ngữ lớn
LSTM	Long Short-Term Memory	Bộ nhớ ngắn hạn dài
MCQ	Multiple-Choice Question	Câu hỏi trắc nghiệm
NLP	Natural Language Processing	Xử lý ngôn ngữ tự nhiên
OCR	Optical Character Recognition	Nhận dạng ký tự quang học
PDF	Portable Document Format	Định dạng tài liệu di động
RAG	Retrieval-Augmented Generation	Sinh tăng cường truy xuất
RLHF	Reinforcement Learning from Human Feedback	Học tăng cường từ phản hồi con người
RNN	Recurrent Neural Network	Mạng nơ-ron hồi quy
sLLM	Small Large Language Model	Mô hình ngôn ngữ lớn cỡ nhỏ
MAS	Multi-Agent System	Hệ thống đa tác tử
VLM	Vision-Language Model	Mô hình thị giác - ngôn ngữ
WER	Word Error Rate	Tỷ lệ lỗi từ

Chương 1

Mở đầu

Chương đầu tiên của khóa luận trình bày bối cảnh và lý do chọn đề tài, cùng với các nội dung liên quan. Mục đầu tiên mô tả vấn đề đang hiện hữu của bài toán số hóa trong lĩnh vực học tập. Hai mục tiếp theo, báo cáo sẽ trình bày tổng quan giải pháp đề xuất và các đóng góp của khóa luận. Mục cuối cùng trình bày sơ bộ về cấu trúc của báo cáo và nội dung chính của từng chương.

1.1 Đặt vấn đề

Tác vụ số hóa tài liệu học tập là một công việc quan trọng trong lĩnh vực giáo dục, đặc biệt trong thời đại công nghệ xuất hiện trong mọi lĩnh vực của đời sống. Đặc điểm của lĩnh vực này là có lượng dữ liệu dồi dào và vô cùng đa dạng như: sách giáo khoa, đề thi và bài giảng. Bên cạnh việc chuyển đổi tài liệu giáo dục từ dạng vật lý sang dữ liệu số, việc sử dụng chúng một cách hữu ích cũng là một bài toán thiết yếu. Bằng cách tạo ra các bài kiểm tra, đặc biệt là bài kiểm tra trắc nghiệm, giáo viên vừa có thể đánh giá học sinh một cách khách quan và tự động, vừa có thể tận dụng nguồn tài nguyên số này.

Tuy nhiên, một vấn đề lớn được đặt ra là: quá trình thiết kế ra một bộ câu hỏi trắc nghiệm, vừa đảm bảo cả chất lượng và số lượng, yêu cầu lượng thời gian và tài nguyên đáng kể từ giáo viên. Chưa kể, bài kiểm tra trắc nghiệm sau đó cần được kiểm tra và rà soát để đảm bảo các tiêu chuẩn giáo dục, như thang đo Bloom [4]. Hai vấn đề kỹ thuật chính mà giáo viên phải đối mặt khi cần chuyển đổi học liệu sang một bài kiểm tra trắc nghiệm là:

- **Khó khăn trong việc số hóa tài liệu từ ảnh chụp:** Các tài liệu giáo dục thực tế thường ở dạng bản in, trang sách, hoặc ảnh chụp từ điện thoại với chất lượng thấp (ảnh nhòe, mờ, độ phân giải thấp). Không chỉ thế, nội dung của chúng còn bao gồm các dạng dữ liệu phức tạp như bảng biểu, đồ thị và công thức toán, gây khó khăn cho các công cụ truyền thống phổ biến như Tesseract [27]. Hậu quả là văn bản được trích xuất ra có độ chính xác kém, thiếu nội

dung và phá vỡ cấu trúc của văn bản; dẫn tới ảnh hưởng nghiêm trọng đến tác vụ sinh bài kiểm tra.

- **Khó khăn trong việc kiểm soát chất lượng câu hỏi:** Các hướng tiếp cận truyền thống đối với bài toán này thường sử dụng các quy tắc lập trình sẵn (rule-based) [13] để tạo ra các câu hỏi trắc nghiệm có mức độ nhận thức đơn giản. Chủ yếu là các câu hỏi về thông tin bề mặt, không yêu cầu suy luận và phân tích sâu, rất rập khuôn và thiếu tự nhiên. Một hướng đi mới là ứng dụng khả năng hiểu văn bản của các mô hình ngôn ngữ lớn, để từ văn bản đầu vào có thể tạo ra các câu hỏi liên quan với chất lượng sư phạm cao. Tuy nhiên, chi phí để sử dụng các mô hình này là rất lớn, đòi hỏi tài nguyên tính toán cao, đặc biệt đối với lượng học liệu khổng lồ của ngành giáo dục. Không chỉ vậy, ngay cả các LLM cũng khó tránh được việc sinh ra các câu hỏi kém chất lượng hoặc ảo giác, nếu không có một quy trình kiểm tra chặt chẽ.

Từ những thách thức trên, một vấn đề lớn được đặt ra là: Làm thế nào có thể phát triển một hệ thống tự động số hóa tài liệu, tạo ra các bài kiểm tra trắc nghiệm có chất lượng cao với chi phí thấp, có thể triển khai trên hạ tầng có sẵn và bảo mật?

1.2 Giải pháp đề xuất

Nhằm giải quyết bài toán trên, khóa luận đề xuất giải pháp kết hợp các mô hình ngôn ngữ cỡ nhỏ mã nguồn mở (sLLM - Small Large Language Model) với kiến trúc đa tác tử (MAS). Các mô hình được sử dụng có số lượng tham số dưới 15 tỷ, điều này giúp chúng có thể chạy trên hạ tầng sẵn có với chi phí thấp. Bên cạnh đó, kiến trúc đa tác tử thiết lập một quy trình tạo bài kiểm tra chặt chẽ từ bước thiết kế câu hỏi đến bước đánh giá và rà soát câu hỏi vừa sinh. Quy trình đề xuất gồm hai giai đoạn chính:

1. **Giai đoạn trích xuất văn bản bằng VLM nhỏ gọn:** Nhận thấy được những hạn chế cố hữu của các công cụ truyền thống như Tesseract, giải pháp sử dụng các mô hình VLM chuyên dụng cỡ nhỏ như DeepSeek OCR v2 [32], PaddleOCR-VL [6]. Các mô hình này đã chứng minh được khả năng trích xuất văn bản một cách chính xác từ ảnh đầu vào, giúp đảm bảo chất lượng của giai đoạn tạo bài kiểm tra.

2. **Giai đoạn sinh câu hỏi trắc nghiệm bằng kiến trúc đa tác tử:** Kiến trúc này tách bài toán thành nhiều bài toán nhỏ hơn được xử lý bởi một tác tử chuyên biệt. Hệ thống LangGraph [15] được sử dụng để điều phối tác tử:

- **Tác tử *Sinh câu hỏi*:** Tạo mới hoặc tạo lại các câu hỏi trắc nghiệm với đầu vào là tài liệu và mức độ nhận thức hệ thống yêu cầu.
- **Tác tử *Phân loại*:** Phân loại và kiểm tra các câu hỏi được sinh ra có đảm bảo đúng mức độ nhận thức theo thang đo Bloom hay không.
- **Tác tử *Học sinh*:** Tác tử này đóng vai là học sinh và sẽ giải các câu hỏi vừa được sinh ra mà không xem đáp án. Quá trình này giúp phát hiện ra các câu hỏi bị đa nghĩa, bẫy logic bất hợp lý, hoặc phương án nhiễu quá dễ nhận biết.
- **Tác tử *Hiệu chỉnh*:** Dựa trên phản hồi từ Tác tử Học sinh, tác tử này tiến hành sửa đổi, tinh chỉnh ngôn từ và logic để câu hỏi đạt chuẩn sự phạm cao nhất.

Văn bản đầu vào được xử lý bởi bốn tác tử với hai vòng lặp kiểm tra và sửa. Nhờ đó, bài toán có thể được giải quyết với chất lượng tiệm cận các LLM với chi phí thấp và có thể triển khai hoàn toàn trên hạ tầng có sẵn và bảo mật nhờ các ưu điểm của sLLM.

1.3 Đóng góp của khóa luận

Qua quá trình thiết kế, thực nghiệm và đánh giá giải pháp, khóa luận này đề xuất bốn đóng góp mang ý nghĩa cả về mặt học thuật lẫn ứng dụng thực tiễn:

- **Đề xuất kiến trúc hệ thống đa tác tử chuyên biệt cho giáo dục:** Khóa luận đã thiết kế và áp dụng thành công kiến trúc đa tác tử vào các sLLM cho bài toán sinh câu hỏi trắc nghiệm. Hệ thống này có thể tự động tạo và đánh giá câu hỏi trắc nghiệm theo thang đo Bloom. Cơ chế vòng lặp kiểm tra được đề xuất để bù đắp giới hạn suy luận của sLLM.
- **Đánh giá thực nghiệm quy mô và toàn diện:** Khóa luận chạy các thực nghiệm trên 12 mô hình khác nhau bao gồm VLM, sLLM mã nguồn mở và các LLM thương mại cho hai bài toán sinh câu hỏi trắc nghiệm và OCR. Bên

cạnh đó, dữ liệu đánh giá của tác vụ OCR được sinh tự động nhờ kỹ thuật mô phỏng nhiễu.

- **Phát triển khung đánh giá chất lượng sư phạm:** Thay vì sử dụng các độ đo truyền thống như BLEU, ROUGE vốn dĩ không phù hợp để đánh giá các bài toán sinh dữ liệu, nghiên cứu đã thiết lập một quy trình đánh giá tự động dựa vào cơ chế LLM-as-a-Judge [17]. Ba tiêu chí sư phạm được sử dụng là Tính giải được (Solvability), Độ bám sát phân loại nhận thức (Alignment), và Chất lượng phương án nhiễu (Distractor Quality).
- **Hiện thực hóa giải pháp qua ứng dụng Web Openlet:** Khóa luận đã xây dựng và triển khai ứng dụng Web nhằm ứng dụng kiến trúc đề xuất vào trong thực tế. Giáo viên có thể tải tài liệu lên để tạo bài kiểm tra và học sinh có thể tham gia bài kiểm tra đã tạo của giáo viên. Nhờ đó, nghiên cứu đã chứng minh tính khả thi của giải pháp đề xuất.

1.4 Cấu trúc khóa luận

Báo cáo sẽ trình bày một cách có hệ thống về giải pháp đề xuất, quy trình đánh giá, kết quả và xây dựng ứng dụng Web Openlet nhằm hiện thực hóa nghiên cứu. Toàn bộ nội dung khóa luận được chia thành 7 chương với bố cục như sau:

- **Chương 1 - Mở đầu:** Đặt vấn đề, phân tích các rào cản trong các phương pháp hiện tại, từ đó đề xuất giải pháp ứng dụng sLLM kết hợp MAS, đồng thời tóm tắt các đóng góp chính của nghiên cứu.
- **Chương 2 - Cơ sở lý thuyết & Các công trình liên quan:** Phần này cung cấp các lý thuyết nền tảng về thang đo Bloom, hệ thống đa tác tử và phương pháp LLM-as-a-Judge được sử dụng. Ngoài ra, các nghiên cứu liên quan cũng sẽ được trình bày để chỉ ra khoảng trống nghiên cứu hiện tại.
- **Chương 3 - Phương pháp đề xuất:** Chương này sẽ trình bày chi tiết về kiến trúc MAS đề xuất kèm theo thiết kế của từng tác tử cũng như một số kỹ thuật mở rộng liên quan.
- **Chương 4 - Phương pháp đánh giá:** Mục tiêu của chương này là trình bày cách khóa luận thiết lập môi trường đánh giá, mô tả bộ dữ liệu đầu vào,

chiến lược tạo ảnh nhiễu và thiết lập các mô hình cơ sở dùng để đánh giá kết quả của kiến trúc đề xuất.

- **Chương 5 - Kết quả:** Các kết quả của thực nghiệm sẽ được đánh giá và phân tích trong chương này. Báo cáo sẽ so sánh năng lực sinh câu hỏi đa cấp độ, chất lượng OCR và chi phí giữa sLLM và LLM thương mại.
- **Chương 6 - Thiết kế và hiện thực ứng dụng Openlet:** Chương này mô tả quy trình xây dựng ứng dụng Web Openlet thực tế. Nội dung sẽ bao gồm sơ đồ ca sử dụng, kiến trúc của phần mềm kèm theo giới thiệu các tính năng cốt lõi của ứng dụng.
- **Chương 7 - Kết luận và Hướng phát triển:** Chương cuối của khóa luận sẽ tập trung vào tổng kết các kết quả quan trọng nhất mà nghiên cứu đạt được cũng như là các hạn chế đang tồn tại và một số định hướng phát triển trong tương lai.

Chương 2

Cơ sở lý thuyết & Các công trình liên quan

Chương này sẽ trình bày hai nền tảng lý thuyết cốt lõi của nghiên cứu là Thang phân loại Bloom và Hệ thống đa tác tử. Ngoài ra, báo cáo sẽ giới thiệu về thư viện LangGraph được sử dụng để triển khai kiến trúc đề xuất kèm theo kỹ thuật sử dụng LLM để đánh giá kết quả (LLM-as-a-Judge). Phần cuối sẽ đi sâu vào các phương pháp hiện tại và chỉ ra những hạn chế cũng như các khoảng trống nghiên cứu còn tồn tại.

2.1 Phân loại Bloom

Để các hệ thống sinh câu hỏi trắc nghiệm tự động có thể vừa đảm bảo các mục tiêu sư phạm vừa không biến thành một cỗ máy tạo câu hỏi ngẫu nhiên, việc thiết kế độ khó cần dựa trên một thang đo đã được kiểm chứng bởi khoa học. Nghiên cứu này tập trung vào khung nhận thức được công nhận rộng rãi nhất trong lĩnh vực giáo dục là **phân loại Bloom**. Thang đo này được nhà tâm lý học và giáo dục *Benjamin Bloom* đề xuất lần đầu vào năm 1956 [4] và được tinh chỉnh bởi *Anderson* và *Krathwohl* vào năm 2001 [2]. Trong nghiên cứu này, nhằm tối ưu hóa và phù hợp với hình thức thi câu hỏi trắc nghiệm đa lựa chọn (MCQ - Multiple-Choice Question), sáu cấp độ ban đầu của thang đo Bloom được phân loại lại thành ba cấp độ cốt lõi là Truy xuất, Suy luận và tổng hợp, và Lập luận phản biện.

2.1.1 Cấp độ 1 - Truy xuất

Cấp độ Truy xuất tương ứng với bậc Nhớ trong thang đo Bloom sửa đổi. Cấp độ này là nền tảng của sự nhận thức, tập trung chủ yếu vào khả năng ghi nhớ cơ học và truy xuất thông tin dài hạn, hoặc nhận dạng một đối tượng, sự kiện được trình bày rõ ràng trong một đoạn văn bản thuộc bài đọc. Quá trình này thường được gọi là nhận thức nông.

Trong quá trình tinh chỉnh các chỉ dẫn cho LLMs sinh câu hỏi ở cấp độ Truy xuất, hệ thống được cài đặt để tập trung vào các chi tiết cụ thể, định nghĩa thuật ngữ và nhận dạng các sự kiện. Đặc thù của MCQ ở dạng này thường xoay quanh các câu hỏi “Ai?”, “Cái gì?”, “Ở đâu?” và “Khi nào?”. Đối với bài toán này, LLMs thể hiện vô cùng xuất sắc trong việc tự động tạo ra các câu hỏi, bởi vì quy trình này cơ bản chỉ đòi hỏi khả năng đối sánh chuỗi và trích xuất các thực thể từ văn bản mà không cần hiểu sâu hay xây dựng bất kỳ chuỗi suy luận logic nào. Không chỉ vậy, các phương án nhiễu ở mức độ này cũng tương đối dễ xây dựng; chúng thường là các thực thể hoặc dữ liệu xuất hiện trong văn bản gốc để làm khó người đọc.

2.1.2 Cấp độ 2 - Suy luận và tổng hợp

Cấp bậc thứ hai bao gồm hai kỹ năng Suy luận và Tổng hợp, đại diện cho các bậc Hiểu, Áp dụng và Sáng tạo trong phân loại Bloom. Ở cấp độ này, thay vì được hỏi các câu hỏi có thể truy xuất trực tiếp từ bề mặt của văn bản, người học sẽ buộc phải xử lý thông tin đã học và ứng dụng lý thuyết đó vào một ngữ cảnh hoàn toàn mới hoặc một tình huống giả định.

Trong bối cảnh bài toán MCQ, Suy luận yêu cầu người học phải có khả năng liên kết các mệnh đề nằm rải rác trong bài đọc để rút ra các kết quả, nhận định không được tác giả viết ra một cách trực tiếp. Đối với Tổng hợp, cấp độ này yêu cầu người học phải chất lọc, sắp xếp lại các yếu tố riêng lẻ thành một góc nhìn bao quát hoặc dự đoán một kết quả tương lai. Bài toán này đòi hỏi cần có các chỉ dẫn cụ thể và hợp lý cho LLMs. Ngoài ra, việc tạo ra các đáp án nhiễu ở cấp độ này cũng là một thách thức kỹ thuật đối với AI: các đáp án sai phải được thiết kế dựa vào sự suy diễn sai lầm về mặt logic hoặc dựa vào việc người đọc hiểu sai một quy luật nào đó để từ đó có thể tạo ra các lựa chọn bẫy hợp lý về mặt trực giác.

2.1.3 Cấp độ 3 - Lập luận phản biện

Cấp độ cuối cùng của nghiên cứu này là Cấp độ Lập luận phản biện, tương ứng với bậc Phân tích và Đánh giá. Trọng tâm của cấp độ nhận thức này là khả năng chia nhỏ một lượng thông tin đồ sộ, phức tạp thành các thành phần nhỏ hơn nhằm phân tích, nhận diện các giả định ngầm và đưa ra những nhận xét, quan điểm khách quan dựa trên sự đối chiếu với các bằng chứng thực tiễn.

Các câu hỏi trắc nghiệm ở cấp độ này thường yêu cầu người học tưởng tượng ra các tình huống kiểm tra nguyên biện logic, yêu cầu họ phân biệt rõ ràng giữa một sự thật khách quan và một quan điểm chủ quan, hoặc được yêu cầu đánh giá độ chính xác, hợp lý của một lập luận khoa học. Thách thức lớn nhất mà các LLM phải đối mặt ở cấp độ này là việc sinh ra các câu hỏi chất lượng cao chỉ có duy nhất một đáp án đúng. Quan trọng hơn cả, phương án nhiễu ở mức độ này không thể chỉ là những thông tin sai lệch thông thường, mà phải là các lập luận, lý lẽ nhìn bề mặt có vẻ cực kỳ thuyết phục nhưng lại chứa các lỗ hổng về mặt logic, từ đó buộc người học phải sử dụng tư duy phản biện cao mới có thể nhận diện được.

2.2 Hệ thống đa tác tử

2.2.1 Khái niệm hệ thống đa tác tử

Kiến trúc đa tác tử là một kỹ thuật hiện đại cho phép các LLM có thể xử lý các bài toán phức tạp mà một LLM duy nhất khó có thể xử lý một cách chính xác và hiệu quả. Nguyên lý cốt lõi của kỹ thuật này là chia nhỏ bài toán ban đầu thành nhiều nhiệm vụ nhỏ hơn và giao cho một mạng lưới các tác tử xử lý. Trong hệ thống này, mỗi tác tử sẽ được chỉ định thực hiện một nhiệm vụ duy nhất. Các tác tử có thể tương tác qua lại với nhau để trao đổi thông tin. Triết lý này khác với việc yêu cầu một LLM giải quyết trọn vẹn một bài toán, một công việc mà đòi hỏi quy trình phức tạp về cả logic và nghiệp vụ, khiến kết quả đầu ra kém chất lượng và có nguy cơ “ảo giác” (hallucination) cao.

Mỗi tác tử trong kiến trúc này được thiết kế để mô phỏng cách mà con người phân công và xử lý công việc. Kỹ thuật **ReAct** [36] thường được sử dụng để yêu cầu các LLM thực hiện một chuỗi hành động lặp lại gồm suy nghĩ, hành động, quan sát kết quả và điều chỉnh suy nghĩ mỗi khi thực hiện một tác vụ. Nhằm tối ưu tài nguyên nhận thức của các LLM, đặc biệt là các sLLM, mỗi tác tử thường sẽ nhận được một chỉ dẫn hẹp với nhiệm vụ hoàn thành một công việc duy nhất. Ví dụ, Tác tử *Sinh câu hỏi* chỉ có nhiệm vụ tạo ra các câu hỏi trắc nghiệm với một cấp độ Bloom nhất định trong khi tác tử *Phân loại* lại có nhiệm vụ đánh giá và dự đoán cấp độ của các câu hỏi vừa tạo, từ đó giúp tác tử *Sinh câu hỏi* có thể điều chỉnh cho phù hợp. Sự kết hợp giữa tính chuyên môn hóa cao và sự tương tác giữa các tác tử giúp cải thiện chất lượng nội dung sinh ra, duy trì độ ổn định và giảm hiện

tượng ảo giác thường thấy ở các LLM.

2.2.2 Thư viện LangGraph

Nghiên cứu sử dụng thư viện LangGraph [15] để triển khai và cài đặt kiến trúc đa tác tử đề xuất một cách đúng đắn và thuận tiện nhất. LangGraph là một thư viện mã nguồn mở chuyên dụng dành cho phát triển các hệ thống LLM và là một thành phần quan trọng trong hệ sinh thái LangChain [5] nổi tiếng. Tuy nhiên, khác với triết lý tuyến tính của LangChain truyền thống, LangGraph sử dụng một đồ thị có hướng để điều phối các tác tử kèm theo một đối tượng trạng thái để lưu trữ dữ liệu. Ba thành phần cốt lõi của một hệ thống đa tác tử khi được triển khai bằng thư viện LangGraph là:

- **Trạng thái:** Đây là một đối tượng lưu trữ dữ liệu tập trung của toàn bộ đồ thị. Nó hoạt động như một bộ nhớ chung nhằm duy trì ngữ cảnh cho hệ thống. Một số loại dữ liệu cần lưu trong cấu trúc dữ liệu này có thể kể đến như danh sách các câu hỏi đã tạo và lỗi của từng câu.
- **Nút:** Đây là nơi dữ liệu được xử lý thông qua các hàm Python. Logic và nghiệp vụ của hệ thống sẽ được triển khai trong các đối tượng Nút này. Quy trình hoạt động bắt đầu từ bước tiếp nhận trạng thái hiện tại, gửi lệnh đến LLM để xử lý và cập nhật đầu ra vào đối tượng Trạng thái chung.
- **Cạnh:** Thành phần này có nhiệm vụ xác định hướng di chuyển của luồng dữ liệu giữa các nút của đồ thị có hướng. Mỗi cạnh có thể bao gồm các điều kiện rẽ nhánh đặc biệt hoặc có khả năng kích hoạt cơ chế thực thi song song. Cạnh sẽ được nối với Nút END mô tả trạng thái kết thúc của luồng dữ liệu.

Nhờ cách thiết kế trên, thư viện LangGraph cho phép các hệ thống đa tác tử có khả năng tự định hướng luồng dữ liệu. Nếu một tác tử *Phân loại* phát hiện ra một câu hỏi sinh ra có cấp độ nhận thức không đúng so với cấu hình, tác tử này có thể yêu cầu tác tử *Sinh câu hỏi* tạo lại câu hỏi đó. Tương tự, nếu tác tử *Học sinh* phát hiện ra một câu hỏi có các đáp án bị lỗi, nó hoàn toàn có thể yêu cầu sửa đổi câu hỏi thông qua tác tử *Hiệu chỉnh*. Các vòng lặp này chỉ dừng lại khi các tác tử kiểm tra chấp nhận kết quả sinh.

2.3 Phương pháp đánh giá bằng LLM

Việc đánh giá chất lượng của dữ liệu tạo sinh luôn là một bài toán phức tạp trong lĩnh vực trí tuệ nhân tạo, đặc biệt là xử lý ngôn ngữ tự nhiên. Các độ đo truyền thống như BLEU hay ROUGE tỏ ra kém hiệu quả trong việc đánh giá chất lượng của LLM vì điểm yếu cố hữu của các độ đo này là không hiểu ngữ nghĩa và ngữ cảnh của dữ liệu. Bên cạnh đó, việc đánh giá bởi con người thường tiêu tốn chi phí, thời gian và hạn chế về khả năng mở rộng mặc dù có độ chính xác cao. Phương pháp sử dụng mô hình ngôn ngữ lớn làm giám khảo (LLM-as-a-Judge) giải quyết vấn đề này bằng cách sử dụng chính các LLM thương mại mạnh mẽ để thay thế vai trò của con người trong việc kiểm tra và đánh giá chất lượng dữ liệu đầu ra.

Liu và các cộng sự đã đề xuất kỹ thuật G-Eval [17] vào năm 2023 để hiện thực hóa triết lý này. Nguyên lý của G-Eval là sử dụng LLM giám khảo nhận đầu vào gồm 3 thành phần: nội dung nguồn, nội dung cần đánh giá và một bộ tiêu chuẩn đánh giá chi tiết. Đầu ra của LLM giám khảo có thể là một nhãn nhị phân đúng sai hoặc một điểm số cụ thể. Phương pháp LLM-as-a-Judge mang lại ba lợi thế quan trọng. Thứ nhất là khả năng mở rộng vượt trội. Hệ thống đánh giá có thể xử lý hàng nghìn dữ liệu trong thời gian ngắn và chi phí thấp hơn rất nhiều so với con người. Thứ hai, phương pháp này cung cấp khả năng thay đổi linh hoạt. Bộ tiêu chuẩn có thể được cập nhật và điều chỉnh dễ dàng. Thứ ba, LLM giám khảo có tính ổn định cao hơn con người, kết quả có thể dễ dàng tái lập, loại bỏ được các yếu tố chủ quan của những người đánh giá khác nhau.

Ngoài ra, kỹ thuật này thường được kết hợp với kỹ thuật chuỗi suy luận (CoT - Chain-of-Thought) [33] được đề xuất bởi *Wei* và các cộng sự vào năm 2022. Kỹ thuật này yêu cầu LLM giám khảo phải đưa ra các lập luận, đánh giá cụ thể trước khi đưa ra điểm số cuối cùng, nhờ đó giúp cải thiện chất lượng và độ ổn định của quy trình đánh giá.

Tuy nhiên, phương pháp LLM-as-a-Judge vẫn tồn tại một số hạn chế cần được lưu ý. Đầu tiên, LLM giám khảo có thể mắc các lỗi “thiên vị” đối với các kết quả được sinh ra bởi chính LLM cùng loại. Không chỉ vậy, nếu không kết hợp với kỹ thuật như CoT, LLM giám khảo có thể đưa ra các kết luận không chính xác. Chính vì vậy, kết quả từ LLM-as-a-Judge nên được đối chiếu với đánh giá từ con người nhằm xác nhận độ tin cậy.

2.4 Các công trình liên quan

Trong phần này, khóa luận sẽ trình bày các hướng nghiên cứu hiện nay về bài toán sinh câu hỏi trắc nghiệm từ văn bản. Về tổng quan, hai phương pháp sinh câu hỏi chính bao gồm: Phương pháp sinh câu hỏi truyền thống và sinh câu hỏi dựa trên mô hình ngôn ngữ lớn.

2.4.1 Sinh câu hỏi trắc nghiệm tự động bằng quy tắc

Trước khi có sự xuất hiện của các mô hình học sâu, các bài toán liên quan đến xử lý ngôn ngữ tự nhiên (NLP - Natural Language Processing) thường được giải quyết bằng các quy tắc ngôn ngữ học được thiết kế một cách thủ công bởi chuyên gia. *Heilman* và *Smith* [13] đã đề xuất phương pháp tiếp cận bằng cách xếp hạng thống kê các câu hỏi được sinh ra. Cụ thể, hệ thống sẽ sinh ra một số lượng lớn các câu hỏi từ văn bản đầu vào thông qua các phép biến đổi về mặt cú pháp. Tiếp theo, hệ thống sẽ sử dụng một mô hình xếp hạng thống kê để lọc và chọn ra các câu hỏi có chất lượng sư phạm tốt nhất.

Với một hướng tiếp cận khác, *Mitkov* và các cộng sự [19] lại đề xuất một hệ thống tạo câu hỏi trắc nghiệm sử dụng các kỹ thuật NLP như nhận dạng thực thể có tên, gán nhãn loại từ và phân tích cú pháp. Nhờ đó, hệ thống có thể nhận diện ra các khái niệm, đối tượng quan trọng trong văn bản đầu vào, từ đó có thể áp dụng trực tiếp các quy tắc biến đổi ngôn ngữ để chuyển thành các câu hỏi và các đáp án nhiều tương ứng.

Mặc dù các phương pháp truyền thống cung cấp cho người dùng khả năng kiểm soát cao về mặt ngữ pháp và cấu trúc câu hỏi, phương pháp này lại có nhiều hạn chế lớn khiến chúng khó có thể ứng dụng trong bài toán thực tế. Thứ nhất, các câu hỏi được tạo ra bằng quy tắc thường có mức độ nhận thức thấp theo thang đo Bloom [4]. Các câu hỏi này chủ yếu biến đổi trực tiếp nội dung trong văn bản, điều này dẫn đến việc chỉ có thể sinh ra các câu hỏi tái hiện thông tin bề mặt, chất lượng sư phạm không cao. Thứ hai, các phương án nhiễu của phương pháp này không gây khó khăn cho học sinh, thường bị loại trừ một cách dễ dàng. Thứ ba, công việc thiết kế ra các bộ quy tắc ngôn ngữ cho hệ thống sinh câu hỏi chuyên môn cao về ngôn ngữ học và tiêu tốn thời gian và tài nguyên.

2.4.2 Sinh câu hỏi trắc nghiệm dựa trên mô hình ngôn ngữ lớn

Sự ra đời của các LLM đã mở ra một hướng đi hoàn toàn mới trong lĩnh vực tạo sinh của ngành giáo dục, đặc biệt là với bài toán tạo câu hỏi trắc nghiệm từ văn bản. Với khả năng tạo ra các văn bản có cấu trúc ngữ pháp và từ vựng chính xác, các hệ thống sinh có thể loại bỏ hoàn toàn các quy tắc ngữ pháp thủ công phức tạp. Không những thế, các LLM còn có khả năng hiểu ngữ cảnh và chỉ dẫn một cách chặt chẽ.

Elkins và cộng sự [7] đã thực hiện các đánh giá chất lượng của các câu hỏi được sinh ra bởi các LLM trong lĩnh vực giáo dục. Kết quả nghiên cứu cho thấy mặc dù các mô hình ngôn ngữ này có thể tạo ra các câu hỏi có chất lượng đạt yêu cầu để đưa vào ứng dụng trong giảng dạy thực tế, mức độ nhận thức của các câu hỏi này chưa cao theo thang đo Bloom. Nghiên cứu này chỉ ra rằng khoảng cách giữa các câu hỏi ghi nhớ bề mặt đơn thuần và câu hỏi yêu cầu phân tích, suy luận vẫn là một thách thức lớn đối với các LLM hiện nay.

2.5 Phân tích khoảng trống nghiên cứu

Từ quá trình tìm hiểu và tổng quát hóa các công trình nghiên cứu liên quan, khóa luận rút ra hai vấn đề hiện hữu của bài toán sinh câu hỏi trắc nghiệm tự động:

Khả năng kiểm soát sự phạm và độ chính xác thấp ở tư duy bậc cao.

Các phương pháp sinh câu hỏi bằng quy tắc truyền thống như phân tích cú pháp bằng NLP hay từ điển WordNet tỏ ra cứng nhắc, không linh hoạt và yêu cầu công sức con người để thiết kế. Mặt khác, việc sử dụng trực tiếp một LLM để tạo ra toàn bộ một bài kiểm tra trắc nghiệm mà thiếu đi các bước đánh giá thường dẫn đến kết quả có chất lượng thấp hoặc gặp tình trạng ảo giác (hallucination).

Tốn kém tài nguyên và rào cản triển khai thực tế. Các hệ thống giáo dục hiện nay thường phụ thuộc quá nhiều vào các LLM thương mại. Mặc dù các mô hình này đem lại chất lượng câu hỏi tương đối tốt, chúng thường đi kèm với chi phí đắt đỏ và độ trễ cao. Ngoài ra, vấn đề bảo mật dữ liệu cũng cần được xem xét, đặc biệt là đối với các hệ thống học liệu bản quyền của các tổ chức giáo dục lớn.

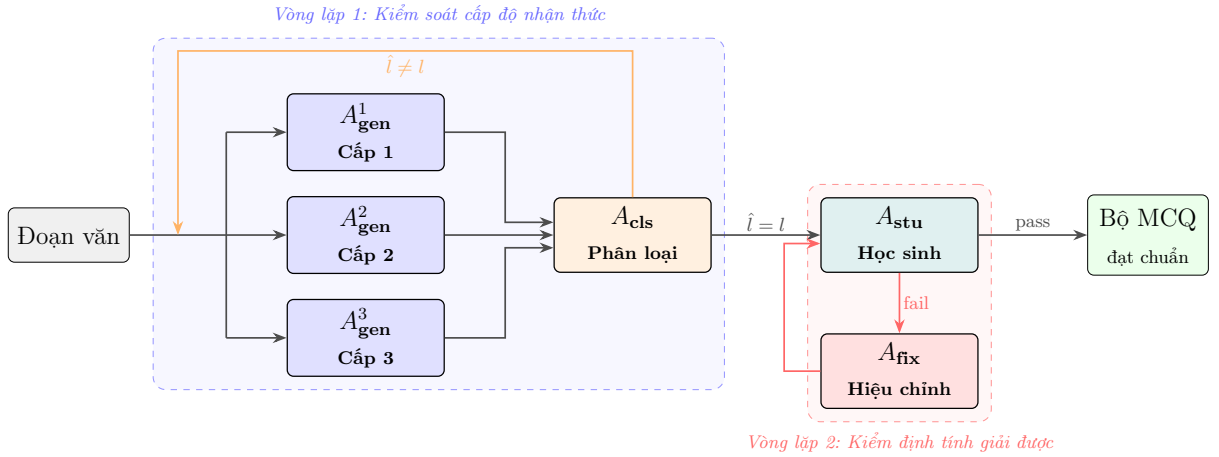
Chương 3

Phương pháp sinh câu hỏi trắc nghiệm bằng luồng đa tác tử

Trong chương này, báo cáo sẽ trình bày chi tiết về hệ thống đa tác tử đề xuất. Mục đầu tiên bao gồm danh sách các tác tử, luồng di chuyển của dữ liệu, các biến trạng thái chung. Trong mỗi mục tiếp theo, báo cáo sẽ đi sâu vào từng tác tử, mô tả dữ liệu đầu vào và đầu ra, kèm theo những quyết định thiết kế nổi bật. Mục cuối sẽ trình bày một số kỹ thuật nghiên cứu sử dụng để cải thiện chất lượng hệ thống.

3.1 Kiến trúc hệ thống

Chất lượng của các câu hỏi trắc nghiệm được sinh ra phụ thuộc trực tiếp vào cách mà văn bản đầu vào được xử lý và di chuyển giữa các tác tử. Trong phần này, báo cáo sẽ trình bày thiết kế của các biến trạng thái được chia sẻ và luồng giao tiếp giữa các tác tử.



Hình 3.1: Kiến trúc hệ thống đa tác tử với hai vòng lặp kiểm soát chất lượng: vòng lặp phân loại cấp độ nhận thức và vòng lặp kiểm định tính giải được của câu hỏi

Toàn bộ luồng suy luận được cài đặt bằng thư viện LangGraph [15] và được điều phối thông qua đối tượng `WorkflowState`. Mỗi trạng thái s sẽ gồm hai nhóm biến trạng thái: các biến đầu vào $\mathcal{I}$ được khởi tạo từ dữ liệu bài đọc và các biến trạng

thái trung gian $\mathcal{O}$ được cập nhật liên tục bởi các tác tử trong quá trình xử lý.

$$\mathcal{I} = \{content, n, max_loops\} \quad (3.1)$$

$$\mathcal{O} = \{questions\} \quad (3.2)$$

Như trong Hình 3.1, hệ thống tác tử hoạt động theo hai vòng lặp kiểm soát chất lượng. Hai vòng lặp này được sắp xếp tuần tự và chuyên biệt cho một phần của câu hỏi: Vòng thứ nhất sẽ cải thiện chất lượng phần thân câu hỏi và Vòng lặp 2 sẽ tập trung cải thiện chất lượng đáp án nhiều.

Vòng lặp 1. Đảm bảo tất cả câu hỏi sinh ra đều có cấp độ nhận thức đúng như yêu cầu đầu vào theo thang đo Bloom [4]. Để làm được việc này, dữ liệu tạo ra từ tác tử *Sinh câu hỏi* sẽ được chuyển cho tác tử *Phân loại* nhằm đánh giá và dự đoán cấp độ từng câu. Nếu tác tử *Phân loại* dự đoán cấp độ khác so với cấp độ được yêu cầu, tác tử *Sinh câu hỏi* sẽ được yêu cầu để tạo loại câu hỏi đó. Ngoài ra, hệ thống được thiết kế với ba tác tử, mỗi tác tử sẽ phụ trách một cấp độ riêng biệt, điều này đảm bảo mỗi tác tử sinh sẽ chuyên biệt hóa cho một cấp độ, từ đó tăng chất lượng đầu ra.

Vòng lặp 2. Tác tử *Học sinh* sẽ được yêu cầu giải các câu hỏi đã qua Vòng lặp 1 mà không được nhìn đáp án. Nếu đáp án từ tác tử *Học sinh* khác so với đáp án được tạo, câu hỏi sẽ bị gán nhãn “mơ hồ” và được chuyển giao cho tác tử *Hiệu chỉnh* nhằm sửa lại đáp án đúng và đáp án gây nhiễu.

3.2 Luồng xử lý đa tác tử

3.2.1 Tác tử *Sinh câu hỏi*

Hệ thống triển khai ba tác tử *Sinh câu hỏi* hoạt động song song và độc lập. Mỗi tác tử sẽ chịu trách nhiệm sinh các câu hỏi trắc nghiệm cho một cấp độ nhận thức $l \in \{1, 2, 3\}$ theo thang đo Bloom. Đầu vào của mỗi tác tử là một đoạn văn bản nguồn, số lượng câu hỏi cần sinh và hai tập các câu hỏi mẫu tham chiếu từ các vòng lặp trước.

$$A_{\text{gen}}^l : (content, n_l, \mathcal{P}_l, \mathcal{N}_l) \longrightarrow \{q_1, q_2, \dots, q_{n_l}\} \quad (3.3)$$

trong đó n_l là số câu hỏi cần sinh cho cấp độ l , $\mathcal{P}_l$ là tập mẫu tích cực và $\mathcal{N}_l$ là tập mẫu tiêu cực. Mỗi câu hỏi q_i được biểu diễn dưới dạng bộ:

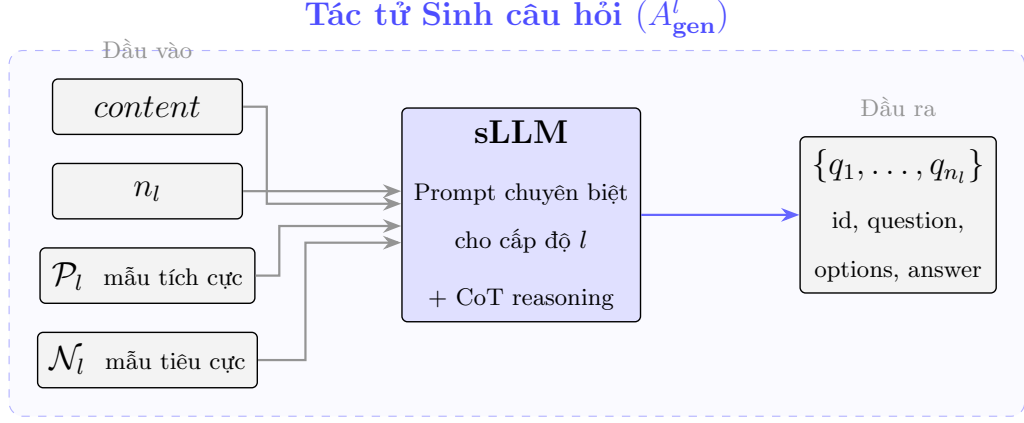
$$q_i = (id, question, options \in \mathbb{S}^4, correct_idx \in \{0, 1, 2, 3\}, level = l, status) \quad (3.4)$$

Mỗi tác tử được thiết kế với một bộ các chỉ dẫn chuyên biệt cho từng cấp độ nhận thức tương ứng. Tác tử *Cấp 1* sẽ được yêu cầu tạo ra các câu hỏi trích xuất thông tin đơn giản, trực tiếp từ văn bản đầu vào. Tác tử *Cấp 2* được yêu cầu sinh ra các câu hỏi suy luận ngầm và đáp án phải được diễn đạt lại hoàn toàn so với văn bản gốc. Cuối cùng, các câu hỏi sinh ra bởi tác tử *Cấp 3* sẽ phải đòi hỏi học sinh thực hiện nhiều bước suy luận và kết nối các thông tin rải rác trong nhiều đoạn văn bản theo chuẩn LSAT/GMAT.

Bên cạnh đó, một kỹ thuật quan trọng được sử dụng trong tác tử *Sinh câu hỏi* là cơ chế học từ các mẫu tham chiếu. Cụ thể, trước khi sinh câu hỏi kể từ vòng lặp thứ hai, tác tử sẽ nhận được hai tập các câu hỏi:

- *Tập các mẫu tích cực*: Đây là danh sách các câu hỏi cùng level đã vượt qua đánh giá của tác tử *Phân loại*. Các câu hỏi này được đảm bảo đã đạt chuẩn về cấp độ nhận thức đúng theo yêu cầu từ hệ thống. Tác tử *Sinh câu hỏi* sẽ học hỏi được phong cách, cấu trúc và độ khó của các mẫu tích cực.
- *Tập các mẫu tiêu cực*: Danh sách này sẽ chứa các câu hỏi có cấp độ khác so với yêu cầu. Các mẫu được lựa chọn chủ yếu là các câu hỏi được tạo lỗi từ các vòng lặp trước. Nếu số mẫu quá ít, hệ thống sẽ lựa chọn các mẫu đã vượt qua đánh giá từ hai cấp độ còn lại. Sử dụng mẫu tiêu cực giúp tác tử tránh lặp lại việc sinh ra các câu hỏi không đạt yêu cầu và phân rõ ranh giới giữa hai cấp độ dễ nhầm lẫn như cấp độ 2 và 3.

Tóm lại, cơ chế của kỹ thuật này hoạt động như một dạng học từ ví dụ (few-shot learning) có phản hồi. Trong đó, chất lượng câu hỏi được cải thiện dần qua các vòng lặp. Không chỉ thế, điều này còn giúp dữ liệu đầu ra dần dần hội tụ về một kết quả, tránh phải chạy quá nhiều vòng lặp không cần thiết. Hình 3.2 minh họa luồng xử lý bên trong tác tử.



Hình 3.2: Luồng xử lý bên trong Tác tử Sinh câu hỏi: nhận văn bản nguồn và mẫu tham chiếu, sinh câu hỏi qua sLLM với prompt chuyên biệt theo cấp độ

3.2.2 Tác tử *Phân loại*

Tác tử *Phân loại* có nhiệm vụ xác định chính xác cấp độ theo thang Bloom [4] của từng câu hỏi sinh ra bởi bộ ba tác tử *Sinh câu hỏi*. Sau đó, tác tử sẽ tiến hành kiểm tra xem kết quả dự đoán của bản thân giống hay khác so với cấp độ được yêu cầu thực tế. Tiếp theo, tác tử *Phân loại* sẽ chuẩn bị danh sách các câu hỏi lỗi cần được sinh lại của từng cấp độ, đồng thời cập nhật các tập tích cực và tiêu cực:

$$A_{\text{cls}} : (content, \{q_1, \dots, q_m\}) \longrightarrow \{(id_i, \hat{l}_i)\}_{i=1}^m \quad (3.5)$$

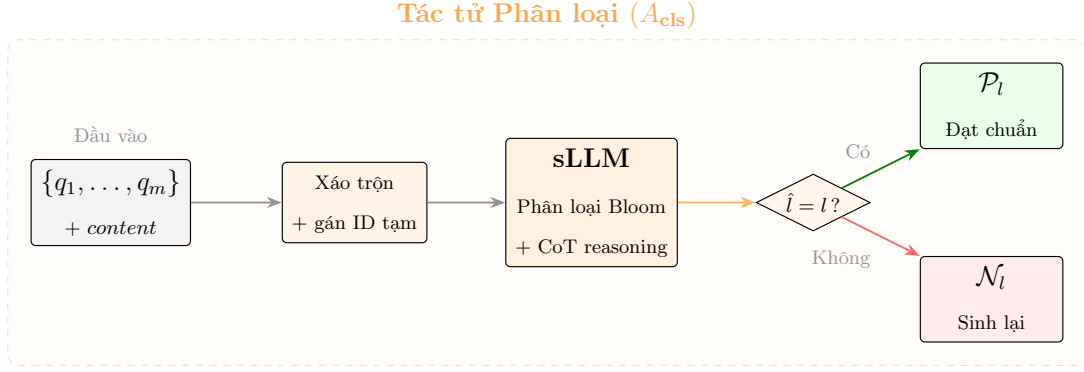
trong đó $\hat{l}_i$ là cấp độ nhận thức do tác tử dự đoán cho câu hỏi q_i .

$$\mathcal{P}_l^{(t+1)} = \mathcal{P}_l^{(t)} \cup \{q.question \mid \hat{l}(q) = l\} \quad (3.6)$$

$$\mathcal{N}_l^{(t+1)} = \mathcal{N}_l^{(t)} \cup \{q.question \mid \hat{l}(q) \neq l\} \quad (3.7)$$

trong đó $\mathcal{P}_l^{(t)}$ và $\mathcal{N}_l^{(t)}$ lần lượt là tập mẫu tích cực và tiêu cực tại vòng lặp t . Câu hỏi đạt chuẩn ($\hat{l} = l$) được chuyển sang Tác tử *Học sinh*; câu hỏi bị từ chối ($\hat{l} \neq l$) được đánh dấu thất bại và nút điều phối sẽ yêu cầu tác tử *Sinh câu hỏi* tạo lại số lượng thiếu hụt. Hình 3.3 minh họa luồng xử lý bên trong tác tử.

Nhằm tiết kiệm chi phí tính toán và tăng tốc độ xử lý, hệ thống sẽ gom toàn bộ câu hỏi sinh ra từ ba tác tử *Sinh câu hỏi* và đưa qua phân loại trong một lần gọi mô hình duy nhất. Tuy nhiên, một vấn đề được đặt ra là khả năng thiên kiến theo



Hình 3.3: Luồng xử lý bên trong Tác tử Phân loại: xáo trộn câu hỏi chống thiên lệch, phân loại cấp độ Bloom, và tách kết quả thành tập đạt chuẩn và tập sinh lại

vị trí của LLM (position bias). Các bằng chứng thực nghiệm đã chỉ ra rằng LLM, kể cả ở quy mô lớn, có thể bị ảnh hưởng bởi thứ tự vật lý của thông tin trong nội dung đầu vào thay vì chỉ dựa trên nội dung ngữ nghĩa của dữ liệu [31]. Ví dụ: LLM có thể phân loại các câu hỏi nằm ở đầu là câu hỏi cấp độ 1 và các câu hỏi ở cuối là cấp độ 3 trong khi cấp độ thực tế có thể là 2. Để giảm ảnh hưởng này, hệ thống sẽ xáo trộn thứ tự của tất cả câu hỏi đầu vào và loại bỏ thông tin về thứ tự câu. Điều này giúp mô hình hạn chế dựa vào vị trí hay thứ tự để tự suy đoán, từ đó tập trung hơn vào nội dung của câu hỏi khi phân loại cấp độ nhận thức.

3.2.3 Tác tử *Học sinh*

Trong vòng lặp tiếp theo, các lựa chọn của các câu hỏi sẽ được kiểm tra và hiệu chỉnh. Để có thể đánh giá chính xác được chất lượng của đáp án đúng và ba đáp án nhiễu, nghiên cứu sẽ mô phỏng lại quá trình một học sinh làm một bài kiểm tra trắc nghiệm trong thực tế bằng cách yêu cầu tác tử giải và đưa ra đáp án.

$$A_{stu} : (content, \{q_1, \dots, q_m\}) \longrightarrow \{(id_i, C_i, reason_i)\}_{i=1}^m \quad (3.8)$$

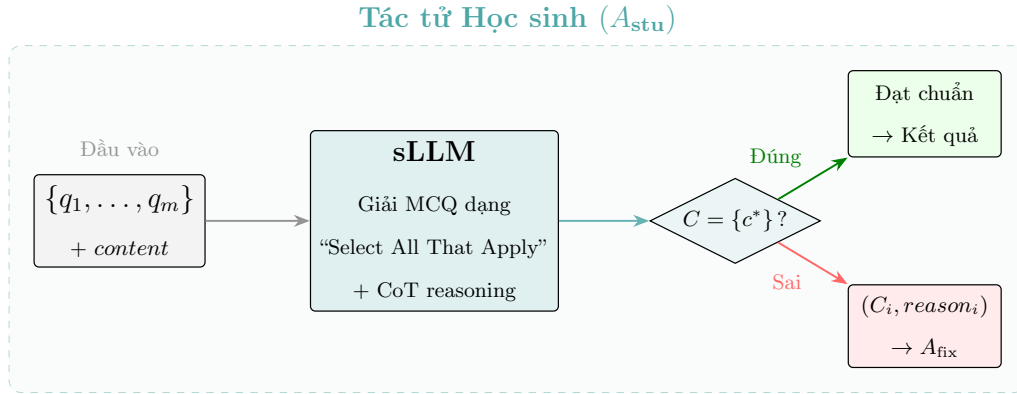
trong đó $C_i \subseteq \{A, B, C, D\} \cup \{\text{NONE}\}$ là tập hợp các phương án được tác tử lựa chọn cho câu hỏi q_i .

Một quyết định thiết kế cốt lõi là việc tác tử *Học sinh* được yêu cầu giải bài kiểm tra theo định dạng trắc nghiệm nhiều lựa chọn đúng (Select All That Apply - SATA). Các nghiên cứu đánh giá đã chỉ ra rằng khi yêu cầu LLM giải một câu hỏi trắc nghiệm một đáp án đúng đơn thuần, mô hình thường sẽ tìm cách loại trừ và

chọn ra đáp án có vẻ hợp lý nhất thay vì thực sự hiểu nội dung của các lựa chọn [34]. Bằng cách “đánh lừa” LLM rằng bài kiểm tra yêu cầu lựa chọn các đáp án đúng, mô hình bị ép buộc phải thẩm định từng phương án một cách độc lập so với các phương án còn lại. Nhờ cơ chế khắt khe này, hệ thống có thể vô hiệu hóa khả năng suy luận đoán mò của tác tử, từ đó giúp phát hiện ra các câu hỏi có các lựa chọn mơ hồ, không chính xác. Nếu tác tử *Học sinh* chọn hai đáp án đúng, hệ thống sẽ đánh dấu câu hỏi đó là mơ hồ, còn nếu tác tử không lựa chọn phương án nào thì câu hỏi sẽ được đánh dấu là không có đáp án đúng. Hay nói cách khác, câu hỏi được xác nhận đạt chuẩn khi và chỉ khi tác tử lựa chọn đúng một phương án duy nhất trùng với đáp án đã sinh từ vòng lặp trước.

$$passed(q) = \begin{cases} true & \text{nếu } C(q) = \{c^*\} \\ false & \text{ngược lại} \end{cases} \quad (3.9)$$

trong đó c^* là ký hiệu phương án đúng của câu hỏi q . Nếu tác tử trả lời sai, câu hỏi được chuyển tới Tác tử *Hiệu chỉnh* cùng với lý do giải thích. Hình 3.4 minh họa luồng xử lý bên trong tác tử.



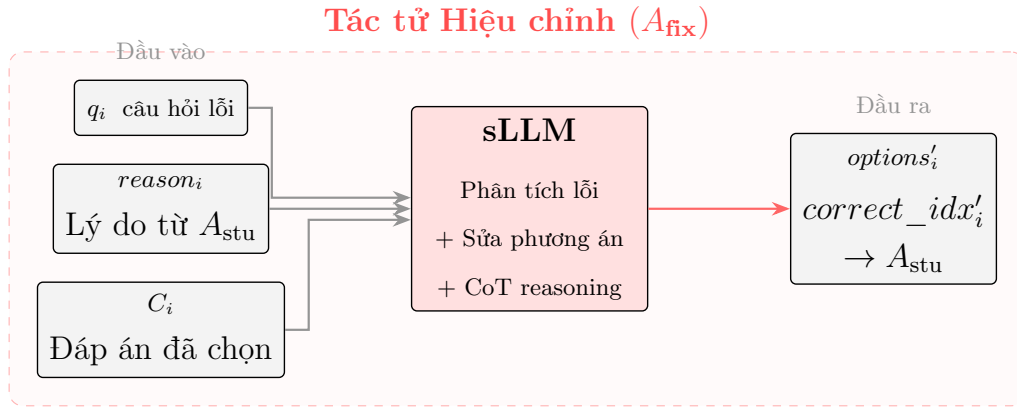
Hình 3.4: Luồng xử lý bên trong Tác tử Học sinh: giải MCQ dạng nhiều lựa chọn đúng, phân luồng câu hỏi đạt chuẩn và câu hỏi cần hiệu chỉnh kèm lý do

3.2.4 Tác tử *Hiệu chỉnh*

Tác tử *Hiệu chỉnh* có nhiệm vụ tiếp nhận và sửa lỗi các câu hỏi lỗi mà tác tử *Học sinh* trả lời sai. Tác tử này sẽ được thiết kế để chỉ được phép chỉnh sửa nội dung của phần lựa chọn mà không được chỉnh sửa phần thân câu hỏi, trong khi bắt buộc giữ nguyên phần thân câu hỏi và cấp độ nhận thức.

$$A_{\text{fix}} : (content, \{(q_i, C_i, reason_i)\}) \longrightarrow \{(id_i, options'_i, correct_idx'_i)\} \quad (3.10)$$

Bên cạnh đó, để tác tử *Hiệu chỉnh* có thể xác định chính xác nguyên nhân và tìm cách sửa lỗi, tác tử sẽ nhận được trường **Reason** từ đầu ra của tác tử *Học sinh*. Trường này mô tả quá trình suy luận của tác tử *Học sinh* bao gồm lý do mà các phương án hoặc không phương án nào được lựa chọn. Không chỉ vậy, tác tử *Hiệu chỉnh* có thể dựa vào đoạn suy luận này để xác định ra các đáp án nhiều quá hiển nhiên nếu nó nhận thấy tác tử *Học sinh* đã loại trừ chúng một cách đơn giản. Sau khi sửa chữa, câu hỏi được đưa trở lại Tác tử *Học sinh* để tái kiểm định. Hình 3.5 minh họa luồng xử lý bên trong tác tử.



Hình 3.5: Luồng xử lý bên trong Tác tử Hiệu chỉnh: nhận câu hỏi lỗi kèm lý do từ Tác tử Học sinh, chỉ sửa phương án và đáp án, đưa lại để tái kiểm định

3.3 Một số kỹ thuật cải thiện chất lượng sLLM

Các sLLM có một nhược điểm cố hữu là khả năng suy luận kém, kèm theo khả năng tuân thủ chỉ thị hạn chế. Điều này có thể ảnh hưởng lớn đến chất lượng của kết quả, đặc biệt là trong kiến trúc đa tác tử, nơi mà dữ liệu của tác tử này tiếp tục được tác tử khác xử lý, từ đó dẫn đến rủi ro tích tụ lỗi.

3.3.1 Suy luận chuỗi tư duy

Trước tiên, để cải thiện chất lượng suy luận của các tác tử trong luồng xử lý, nghiên cứu áp dụng kỹ thuật suy luận chuỗi tư duy (CoT - Chain-of-Thought [33]). Cụ

thể, mỗi LLM được yêu cầu sinh ra một đoạn suy luận ngắn trước khi tạo ra bất kỳ dữ liệu nào. Đoạn suy luận này sẽ nêu dẫn chứng và lý do mô hình tạo ra các dữ liệu tiếp theo:

- **Tác tử *Sinh câu hỏi*:** Trường Reason yêu cầu mô hình giải thích chiến lược tạo câu hỏi và các phương án trước khi sinh nội dung.
- **Tác tử *Phân loại*:** Trường Reason yêu cầu mô hình phân tích nội dung câu hỏi dựa trên đặc điểm của cấp độ nhận thức Bloom trước khi đưa ra dự đoán.
- **Tác tử *Học sinh*:** Trường Reason yêu cầu mô hình trình bày chứng minh từng bước trong ba câu, giải thích lý do chọn hoặc loại từng phương án. Trường này sẽ được sử dụng bởi tác tử *Hiệu chỉnh* trong pha tiếp theo.
- **Tác tử *Hiệu chỉnh*:** Trường Reason yêu cầu mô hình phân tích nguyên nhân lỗi từ phần suy luận của tác tử *Học sinh* trước khi chỉnh sửa các phương án.

Kỹ thuật CoT đặc biệt quan trọng đối với các sLLM bởi các mô hình này thường gặp khó khăn trong các tác vụ đòi hỏi suy luận đa bước (tác tử *Học sinh* trả lời câu hỏi cấp độ 3). Bằng cách yêu cầu mô hình trình bày rõ ràng quá trình tư duy trước khi đưa ra một kết quả cụ thể, kỹ thuật này giúp giảm thiểu đáng kể lỗi logic, tăng tính nhất quán và độ ổn định của đầu ra, đồng thời cho phép truy vết và sửa lỗi khi cần thiết.

3.3.2 Định dạng đầu ra của tác tử

Một vấn đề khác xuất hiện khi làm việc với các sLLM là khả năng tuân thủ theo chỉ dẫn kém. Điều này dẫn đến việc các mô hình thường xuyên đưa ra đầu ra với định dạng không đúng như yêu cầu. Đặc biệt đối với định dạng JSON, mô hình thường xuyên gặp các lỗi cú pháp như thiếu dấu ngoặc, ký tự thoát, dẫn đến việc phải tiến hành quy trình sửa lỗi phức tạp hoặc phải chạy lại LLM nếu gặp lỗi nghiêm trọng [8].

Để giải quyết vấn đề này, nghiên cứu sử dụng định dạng văn bản thuần tuân tự đơn giản với cấu trúc *khóa-giá trị*. Quyết định này giúp mô hình dễ dàng hơn trong việc xác định định dạng đầu ra, từ đó giải phóng tài nguyên nhận thức để tập trung suy luận logic thay vì phải đảm bảo đầu ra là JSON hợp lệ. Việc sử dụng

định dạng *khóa-giá trị* còn giúp hệ thống có khả năng chống chịu lỗi tốt hơn khi gặp các lỗi như thừa thiếu khoảng cách hoặc chứa các kí tự đặc biệt. Không chỉ vậy, việc yêu cầu sLLM sinh ra thứ tự của câu hỏi còn giúp mô hình kiểm soát số lượng câu hỏi hiện tại, tránh việc sinh thừa hay thiếu. Hình 3.6 minh họa cấu trúc định dạng đầu ra chuẩn của tác tử.

```
ID: [ID]
Reason: [...]
Question: [text]
A: [option A]
B: [option B]
C: [option C]
D: [option D]
Answer: [A|B|C|D]
```

Hình 3.6: Cấu trúc định dạng đầu ra chuẩn của tác tử *Sinh câu hỏi*

Chương 4

Phương pháp đánh giá

Trong chương này, báo cáo sẽ trình bày chi tiết về thiết kế thực nghiệm nhằm kiểm chứng sự hiệu quả của phương pháp đã đề xuất. Nội dung của chương sẽ bao gồm giới thiệu về bộ dữ liệu được sử dụng trong thực nghiệm, quy trình tạo dữ liệu ảnh nhiễu tổng hợp cho bài toán OCR, cấu hình chi tiết của hệ thống và mô tả về các độ đo được sử dụng.

4.1 Bộ dữ liệu

RACE (ReAding Comprehension Dataset From Examinations) [14] là một bộ dữ liệu hỏi đáp có quy mô lớn trong lĩnh vực xử lý ngôn ngữ tự nhiên. Khác với các bộ dữ liệu hỏi đáp khác như **SQuAD** [25] được trích xuất chủ yếu từ Wikipedia, RACE được xây dựng trực tiếp từ bộ đề thi của các kỳ thi tiếng Anh thực tế dành cho học sinh trung học cơ sở và trung học phổ thông tại Trung Quốc.

Nghiên cứu sử dụng RACE để làm bộ dữ liệu đánh giá bởi hai lý do chính. Thứ nhất, các văn bản trong RACE bao trùm nhiều lĩnh vực trong đời sống như giáo dục, văn hóa, chính trị và khoa học. Nhờ đó, văn phong và đặc điểm dữ liệu của RACE rất giống với dữ liệu thực tế. Theo thống kê, các văn bản trong bộ dữ liệu này có độ dài từ 300 đến 500 từ. Số lượng này đủ lớn để chứa được lượng thông tin phục vụ cho một bài kiểm tra như các chuỗi sự kiện phức tạp, luận điểm trong khi vẫn đảm bảo không gây tràn cửa sổ ngữ cảnh (context window) của các sLLM.

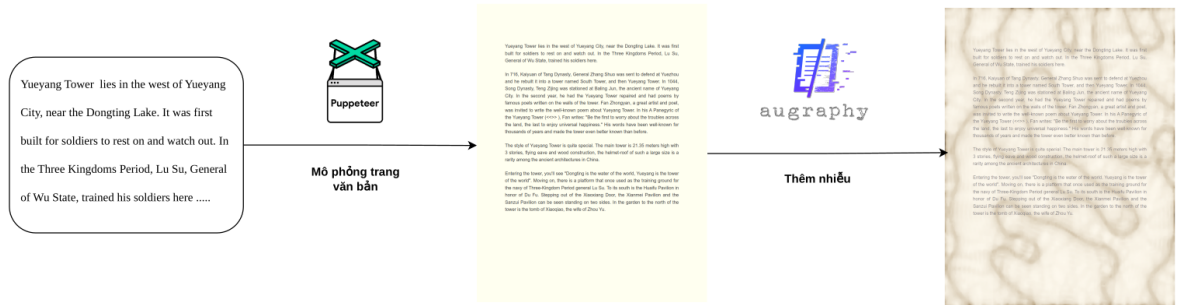
Trong phạm vi thực nghiệm, nghiên cứu lấy mẫu ngẫu nhiên 100 đoạn văn có độ dài lớn nhất từ tập con RACE-High làm dữ liệu đầu vào cho toàn bộ các thí nghiệm sinh câu hỏi. Bảng 4.1 trình bày thống kê tổng quan về bộ dữ liệu RACE và mẫu được sử dụng.

Đặc điểm	RACE-Middle	RACE-High	Mẫu thực nghiệm
Số đoạn văn	~6.400	~21.600	100
Số câu hỏi	~25.400	~72.000	-
Nguồn gốc	Đề thi THCS	Đề thi THPT	Lấy mẫu từ RACE-High
Độ dài trung bình (từ)	250-350	300-500	300-500

Bảng 4.1: Thống kê bộ dữ liệu RACE và mẫu thực nghiệm

4.2 Quy trình tổng hợp bộ dữ liệu OCR

Bên cạnh bài toán sinh câu hỏi trắc nghiệm từ văn bản, nghiên cứu cần xây dựng một bộ dữ liệu nhằm đánh giá chất lượng của các VLM trong tác vụ OCR. Một thách thức lớn được đặt ra là các bộ dữ liệu OCR hiện tại không chuyên biệt cho các văn bản dài. Bên cạnh đó, việc thu thập và gán nhãn thủ công các ảnh chụp văn bản học liệu vô cùng tốn kém về chi phí và thời gian. Để giải quyết vấn đề, nghiên cứu triển khai một quy trình sinh dữ liệu tổng hợp dành riêng cho các ảnh chụp văn bản dài trong lĩnh vực giáo dục. Cụ thể, quy trình này sẽ sử dụng các văn bản từ bộ RACE để kết xuất thành các hình ảnh tương tự ảnh chụp màn hình. Sau đó các ảnh chụp này sẽ được thêm các nhiễu bằng cách mô phỏng tác nhân từ môi trường thực tế. Quy trình hai giai đoạn này được biểu diễn trong Hình 4.1.

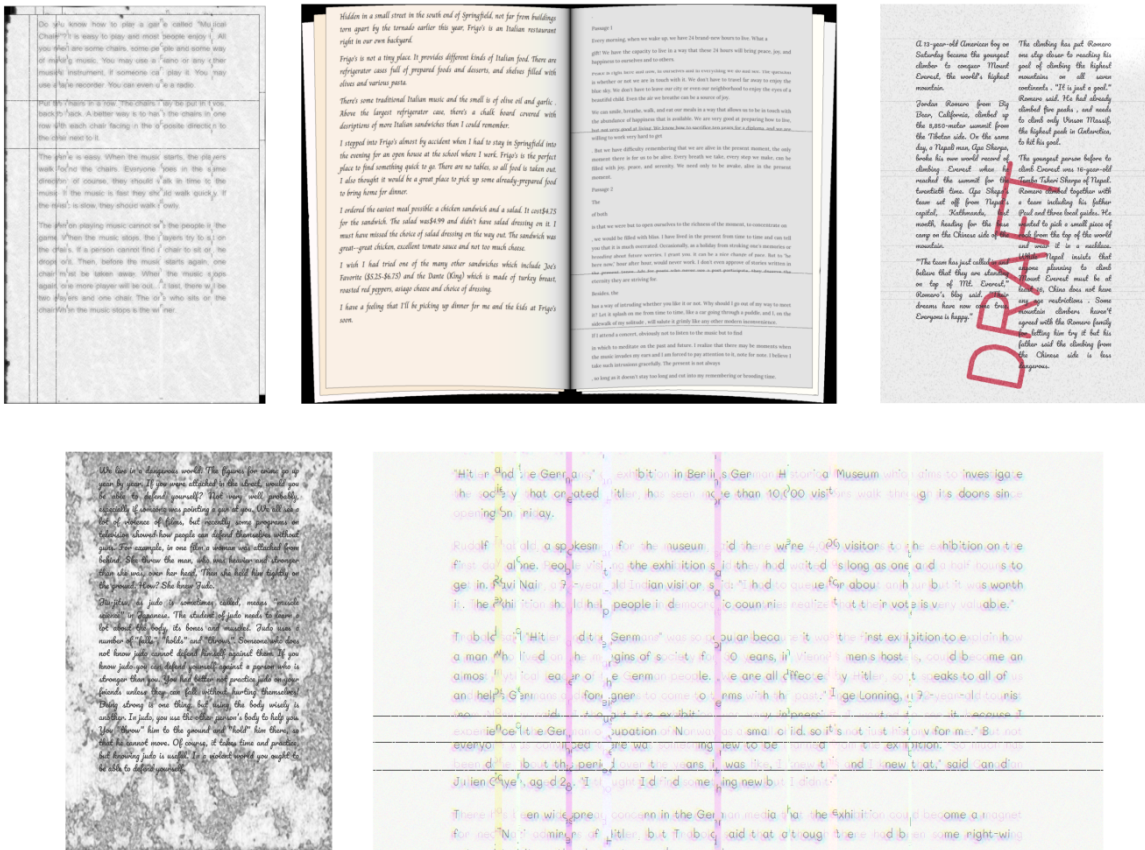


Hình 4.1: Sơ đồ quy trình tăng cường dữ liệu ảnh nhiễu: từ văn bản gốc qua kết xuất ảnh sạch, áp dụng nhiễu vật lý và quang học, đến ảnh giả lập tài liệu thực tế

Giai đoạn sinh ảnh sạch. Hệ thống tạo ra các hình ảnh kỹ thuật số sắc nét từ văn bản đầu vào bằng cách sử dụng trình duyệt không giao diện (headless browser) từ thư viện Pypeteer [20] để tận dụng sự linh hoạt của HTML và CSS. Quy trình này gồm 3 bước cốt lõi: Lựa chọn ngẫu nhiên một bộ cài đặt gồm kích thước ảnh,

màu sắc, phong chữ (kết hợp cả phong chữ in chuẩn và viết tay từ Google Fonts [9]); tự động phân bố mật độ hiển thị của văn bản như số cột, căn lề, khoảng cách giữa các dòng; và cuối cùng là những mã JavaScript để tự động cơ giãn cỡ chữ để toàn bộ văn bản nằm trọn trong khung hình.

Giai đoạn mô phỏng ảnh nhiễu. Nghiên cứu sử dụng thư viện Augraphy [12] để triển khai quy trình tăng cường dữ liệu trên các hình ảnh sạch. Quá trình này tích hợp đa dạng các lớp biến đổi phức tạp bao gồm nhiễu do vật liệu giấy và thiết bị in ấn (vết mực nhòe, đứt nét, giấy óng vàng, chữ in chìm mặt sau), các nhiễu quang học phổ biến trong thực tế (phơi sáng, bóng đổ, lóa sáng) cũng như các nhiễu cơ học (giấy nhăn, gạch chân thủ công, lỗi nén ảnh). Bằng cách mô phỏng rất nhiều biến thể của nhiễu, nghiên cứu có thể đánh giá được khả năng trích xuất văn bản của các VLM trong các điều kiện thực tế khác nhau. Hình 4.2 trình bày một số ảnh được làm nhiễu điển hình được sử dụng thực nghiệm.



Hình 4.2: Các mẫu ảnh tài liệu tổng hợp minh họa các loại nhiễu được áp dụng: nhiễu vật liệu (vết óng vàng, nhòe mực), nhiễu quang học (bóng đổ, nhiễu hạt), và biến dạng hình học (cong vênh trang)

4.3 Thiết lập thực nghiệm

Nghiên cứu đánh giá tổng cộng 12 mô hình được chia thành ba nhóm: nhóm các sLLM mã nguồn mở dùng để sinh câu hỏi trắc nghiệm, nhóm VLM quy mô nhỏ dùng cho tác vụ OCR và nhóm các LLM thương mại dành cho cả hai tác vụ.

Bảng 4.2 tóm tắt các mô hình được sử dụng trong thực nghiệm. Các mô hình sLLM có kích thước tham số từ 8 đến 14 tỷ, phù hợp với mục tiêu triển khai trên phần cứng phổ thông. Các mô hình thương mại được đưa vào làm mô hình cơ sở để đánh giá mức độ cạnh tranh của giải pháp MAS kết hợp sLLM.

Nhóm	Mô hình	Tác vụ
sLLM mã nguồn mở	Gemma 3 12B IT [29]	Sinh câu hỏi
	Qwen3 8B [35]	Sinh câu hỏi
	Phi-4 [1]	Sinh câu hỏi
	Nemotron Nano 9B v2 [23]	Sinh câu hỏi
	Llama 3.1 8B Instruct [11]	Sinh câu hỏi
VLM nhỏ gọn	DeepSeek OCR 2 [32]	OCR
	DotOCR 1.5 [16]	OCR
	MinerU 2.5 [21]	OCR
	PaddleOCR VL 1.5 [6]	OCR
LLM thương mại	Gemini 3 Flash [28]	Sinh câu hỏi, OCR, đánh giá
	GPT-5 Mini [26]	Sinh câu hỏi, OCR
	Claude Haiku 4.5 [3]	Sinh câu hỏi, OCR

Bảng 4.2: Danh sách 12 mô hình sử dụng trong thực nghiệm

Toàn bộ mã nguồn được viết bằng ngôn ngữ Python phiên bản 3.12. Thư viện **LangGraph** [15] được sử dụng để cài đặt và triển khai kiến trúc đa tác tử một cách hiệu quả. Bên cạnh đó, để thuận tiện cho quá trình chạy thực nghiệm và chuyển đổi giữa các mô hình, nghiên cứu sử dụng dịch vụ API từ OpenRouter [24] và NovitaAI [22] để truy cập đến các mô hình sLLM, VLM và LLM thương mại.

Các siêu tham số chính được thiết lập như sau:

- Nhiệt độ (temperature): 0,0 cho các tác tử chính (tác tử *Sinh câu hỏi*, tác tử

Phân loại, tác tử *Hiệu chỉnh*) nhằm đảm bảo tính nhất quán và tái lập được của kết quả; 0,3 cho tác tử *Học sinh* nhằm tạo sự đa dạng trong cách trả lời.

- Số token tối đa (max_tokens): 8.192 token cho mỗi lần gọi mô hình.
- Số vòng lặp tối đa (max_loops): Giới hạn ở 8 vòng cho mỗi chu trình phản hồi giữa các tác tử.

Về quy mô của thực nghiệm, hệ thống được yêu cầu sinh 5 câu hỏi cho mỗi cấp độ nhận thức, tổng cộng là 15 câu hỏi cho mỗi đoạn văn đầu vào. Với 100 đoạn văn mẫu lấy ngẫu nhiên từ bộ RACE, mỗi mô hình cần sinh ra 1.500 câu hỏi.

Để đánh giá hiệu quả của kiến trúc đa tác tử được đề xuất trong bài toán sinh câu hỏi trắc nghiệm, nghiên cứu tiến hành thiết lập hai kịch bản thử nghiệm trên mỗi mô hình sLLM: phương pháp cơ sở, yêu cầu mô hình tạo toàn bộ bài kiểm tra trong một lần gọi duy nhất (single prompt), và phương pháp tích hợp đa tác tử. Riêng đối với các LLM thương mại, nhằm đảm bảo tính công bằng về mặt tài nguyên đánh giá, nghiên cứu chỉ áp dụng cơ chế gọi mô hình một lần. Quyết định này được đưa ra dựa trên thực tế rằng chi phí vận hành cho một lượt truy vấn đối với các LLM thương mại thường tương đương, hoặc thậm chí vượt trội hơn so với tổng chi phí triển khai hệ thống sLLM đa tác tử.

Đối với quy trình LLM-as-a-Judge, nghiên cứu sử dụng Gemini 3 Flash làm mô hình giám khảo cho tất cả thí nghiệm. Lý do nghiên cứu chọn mô hình này dựa vào ba tiêu chí chính: chất lượng suy luận cao cho các tác vụ đánh giá, tốc độ xử lý nhanh và chi phí hợp lý cho việc đánh giá quy mô lớn.

4.4 Phương pháp và Tiêu chí Đánh giá

Mục này sẽ trình bày chi tiết các độ đo được sử dụng trong nghiên cứu. Hệ thống triển khai hai khâu đánh giá độc lập cho hai bài toán tương ứng: đánh giá chất lượng câu hỏi trắc nghiệm và đánh giá độ chính xác của tác vụ OCR.

4.4.1 Đánh giá chất lượng câu hỏi trắc nghiệm

Đối với bài toán sinh câu hỏi trắc nghiệm, các thang đo NLP truyền thống như BLEU hay ROUGE tỏ ra kém hiệu quả và thiếu tin cậy do điểm yếu cốt lõi là chỉ

so sánh sự trùng lặp bề mặt mà thiếu sự thấu hiểu về ngôn ngữ và ngữ cảnh. Bằng cách áp dụng kỹ thuật LLM-as-a-Judge, hệ thống có thể đánh giá một cách chính xác và khách quan các tiêu chí sơ phạm. Cụ thể, nghiên cứu sử dụng ba tiêu chí sơ phạm sau:

Khả năng giải quyết dựa trên ngữ cảnh (Solvability). Tiêu chí này kiểm định khả năng câu hỏi có thể được giải quyết với văn bản đầu vào. Một câu hỏi đạt chuẩn tiêu chí này buộc phải được trả lời dựa vào nội dung của văn bản thay vì thông tin bên ngoài. Không chỉ vậy, câu hỏi phải có đúng một đáp án đúng duy nhất. Để tự động đánh giá độ đo này, nghiên cứu sử dụng LLM thương mại với chất lượng cao để trực tiếp giải quyết câu hỏi. Nếu kết quả của giám khảo giống như kết quả đã sinh, câu hỏi sẽ được đánh dấu đạt chuẩn và ngược lại.

$$\text{Solvability}(q) = \begin{cases} 1, & \text{nếu } C_{\text{judge}}(q) = \{c^*\} \\ 0, & \text{ngược lại} \end{cases}$$

trong đó $C_{\text{judge}}(q)$ là tập hợp các phương án được LLM giám khảo chọn cho câu hỏi q , và c^* là đáp án đúng duy nhất. Nếu chọn sai hoặc chọn nhiều phương án, câu hỏi bị đánh giá 0 (có thể do mơ hồ hoặc chứa nhiều đáp án đúng).

Độ bám sát phân loại nhận thức (Alignment): Tiêu chí này kiểm tra mức độ nhận thức hay độ khó của câu hỏi sinh ra có đúng với yêu cầu của hệ thống từ trước hay không. LLM giám khảo sẽ đối chiếu câu hỏi sinh ra với các tiêu chí của thang đo Bloom, từ đó đưa ra cấp độ thực tế của câu hỏi. Tương tự như tác tử *Phân loại*, câu hỏi đạt tiêu chí này nếu cấp độ giám khảo dự đoán trùng với cấp độ đã tạo.

$$\text{Alignment}(q) = \begin{cases} 1, & \text{nếu } l_{\text{judge}}(q) = l_{\text{target}}(q) \\ 0, & \text{ngược lại} \end{cases}$$

trong đó $l_{\text{judge}}(q)$ là cấp độ do LLM dự đoán và $l_{\text{target}}(q)$ là cấp độ yêu cầu.

Chất lượng phương án nhiễu (Distractor Quality): Tiêu chí này đánh giá chất lượng của ba phương án nhiễu trong mỗi câu hỏi. Một phương án sai hợp lệ phải thỏa mãn đồng thời hai tiêu chí: trái ngược hoàn toàn so với đáp án đúng (không tạo ra sự mơ hồ hay đa nghĩa cho người học) và có khả năng đánh lừa những học sinh không nắm vững kiến thức. LLM giám khảo sẽ đánh dấu các phương án nhiễu hợp lệ, giá trị của độ đo sẽ là tổng số phương án nhiễu hợp lệ trung bình đã được chuẩn hóa.

Gọi $v(q)$ là số phương án nhiều hợp lệ của câu hỏi q (với $v(q) \in \{0, 1, 2, 3\}$). Chỉ số chất lượng cho mỗi cấp độ l được chuẩn hóa về khoảng $[0, 1]$:

$$DQ_l = \frac{1}{N_l} \sum_{q \in \mathcal{A}_l} \frac{v(q)}{3}$$

trong đó $\mathcal{A}_l$ là tập hợp các câu hỏi đã được chấp nhận ở cấp độ l , và $N_l = |\mathcal{A}_l|$.

Chỉ số Distractor Quality tổng thể là trung bình cộng trên ba cấp độ:

$$DQ_{\text{overall}} = \frac{1}{3} \sum_{l=1}^3 DQ_l$$

Trong quá trình thực nghiệm, nghiên cứu phát hiện ra rằng một số mô hình yếu có xu hướng tạo ra các câu hỏi dễ (thường là cấp độ 1 và 2), dẫn đến việc kết quả *Alignment* thấp trong khi *Solvability* lại cao. Hay nói cách khác, các mô hình này sinh ra các câu hỏi có chất lượng tốt về hình thức (đúng một đáp án, không mơ hồ, đúng ngữ cảnh) nhưng không đúng cấp độ nhận thức yêu cầu. Để các đánh giá được công bằng và hiệu quả, nghiên cứu đề xuất độ đo Tỷ lệ chấp nhận (*Acceptance Rate*) là tỷ lệ các câu hỏi đồng thời thỏa mãn cả hai tiêu chí *Solvability* và *Alignment*. Đây cũng là độ đo chính được sử dụng trong báo cáo này.

$$\text{Acceptance}(q) = \text{Solvability}(q) \times \text{Alignment}(q)$$

$$\text{Acceptance Rate} = \frac{1}{N} \sum_{i=1}^N \text{Acceptance}(q_i)$$

trong đó N là tổng số câu hỏi được đánh giá. Chỉ những câu hỏi có *Acceptance* bằng 1 mới được đưa vào Giai đoạn 2.

4.4.2 Đánh giá hiệu năng nhận dạng văn bản

Bài toán thứ hai cần được đánh giá của khóa luận là tác vụ OCR. Nghiên cứu sử dụng hai thang đo tiêu chuẩn là tỷ lệ lỗi ký tự (CER - Character Error Rate) và tỷ lệ lỗi từ (WER - Word Error Rate). Nguyên lý của hai độ đo này là dựa trên khoảng cách chỉnh sửa Levenshtein. Cụ thể, giá trị của độ đo là số các bước sửa đổi tối thiểu để biến đổi chuỗi dự đoán thành chuỗi tham chiếu gốc với ba thao tác chèn (I), xóa (D) và thay thế (S).

$$\text{CER} = \frac{S_c + D_c + I_c}{N_c}$$

$$\text{WER} = \frac{S_w + D_w + I_w}{N_w}$$

trong đó N_c và N_w lần lượt là tổng số ký tự và tổng số từ trong chuỗi tham chiếu. Giá trị CER và WER càng thấp thể hiện chất lượng nhận dạng OCR càng tốt.

Chương 5

Kết quả

5.1 Kết quả sinh câu hỏi

Phần này trình bày kết quả đánh giá chất lượng câu hỏi trắc nghiệm được sinh bởi các mô hình theo ba nhóm: sLLM cơ sở (không có hệ thống đa tác tử), sLLM kết hợp đa tác tử, và LLM thương mại. Ba chỉ số đánh giá chính được sử dụng là **Acceptance** (tỷ lệ câu hỏi được chấp nhận), **Alignment** (mức độ phù hợp giữa cấp độ nhận thức sinh ra và cấp độ yêu cầu), và **Distractor Quality** (chất lượng các phương án nhiễu). Các chỉ số này được đo bằng phương pháp G-Eval với thang điểm từ 0 đến 1. Kết quả được phân tích theo ba cấp độ nhận thức: Cấp 1 (Nhớ), Cấp 2 (Hiểu) và Cấp 3 (Phân tích).

5.1.1 Kết quả nhóm sLLM cơ sở

Bảng 5.1 trình bày kết quả đánh giá của năm mô hình sLLM khi hoạt động độc lập, không có sự hỗ trợ của hệ thống đa tác tử.

Bảng 5.1: Kết quả đánh giá nhóm sLLM cơ sở trên bộ dữ liệu RACE-High

Mô hình	Cấp 1 - Nhớ			Cấp 2 - Hiểu			Cấp 3 - Phân tích		
	Acc.	Align.	Dist.	Acc.	Align.	Dist.	Acc.	Align.	Dist.
Gemma 3 12B IT	0,9080	0,9600	0,8740	0,7780	0,8700	0,6747	0,5160	0,7300	0,4993
Qwen3 8B	0,8300	0,9040	0,8140	0,4040	0,5100	0,2520	0,1192	0,2141	0,0953
Phi-4	0,6018	0,8407	0,4860	0,2808	0,4212	0,1680	0,1187	0,2348	0,0747
Nemotron Nano 9B v2	0,7600	0,8460	0,7347	0,4840	0,5940	0,3567	0,0887	0,1734	0,0747
Llama 3.1 8B Instruct	0,7640	0,8940	0,7100	0,3000	0,4720	0,1893	0,0200	0,0620	0,0133

Kết quả cho thấy Gemma 3 12B IT dẫn đầu toàn diện trên cả ba cấp độ và ba chỉ số đánh giá. Ở Cấp 1, hầu hết các mô hình đạt kết quả khá tốt với Acceptance trên 0,60 và Alignment trên 0,84, cho thấy các sLLM có khả năng sinh câu hỏi ở mức nhận thức thấp một cách đáng tin cậy.

Tuy nhiên, xu hướng sụt giảm chất lượng khi cấp độ nhận thức tăng là rất rõ

rệt. Từ Cấp 1 đến Cấp 3, Acceptance của Qwen3 8B giảm từ 0,8300 xuống 0,1192 (giảm 85,6%), Nemotron Nano 9B v2 giảm từ 0,7600 xuống 0,0887 (giảm 88,3%), và Llama 3.1 8B Instruct giảm từ 0,7640 xuống 0,0200 (giảm 97,4%). Đặc biệt, Llama 3.1 8B Instruct gần như thất bại hoàn toàn ở Cấp 3 với Distractor Quality chỉ đạt 0,0133, cho thấy mô hình này không có khả năng tạo ra các phương án nhiễu có chất lượng cho câu hỏi phân tích.

Kết quả này xác nhận giả thuyết rằng các sLLM khi hoạt động độc lập gặp khó khăn lớn trong việc sinh câu hỏi đòi hỏi tư duy bậc cao, đặc biệt là ở cấp độ Phân tích. Điều này tạo động lực mạnh mẽ cho việc áp dụng kiến trúc đa tác tử nhằm bù đắp hạn chế nội tại của từng mô hình đơn lẻ.

5.1.2 Kết quả nhóm sLLM kết hợp đa tác tử

Bảng 5.2 trình bày kết quả khi cùng năm mô hình sLLM được tích hợp vào hệ thống đa tác tử, nơi các tác tử *Phân loại*, *Học sinh* và *Hiệu chỉnh* cùng phối hợp để nâng cao chất lượng câu hỏi.

Bảng 5.2: Kết quả đánh giá nhóm sLLM đa tác tử trên bộ dữ liệu RACE-High

Mô hình	Cấp 1 - Nhớ			Cấp 2 - Hiểu			Cấp 3 - Phân tích		
	Acc.	Align.	Dist.	Acc.	Align.	Dist.	Acc.	Align.	Dist.
Gemma 3 12B IT	0,9580	0,9940	0,9333	0,8380	0,9600	0,7800	0,7360	0,9920	0,6600
Qwen3 8B	0,9093	0,9897	0,8593	0,8577	0,9753	0,7107	0,5814	1,0000	0,5073
Phi-4	0,9753	0,9959	0,9407	0,9072	0,9711	0,7900	0,6845	1,0000	0,6187
Nemotron Nano 9B v2	0,9232	0,9778	0,8960	0,7818	0,9212	0,7033	0,5556	0,9818	0,4973
Llama 3.1 8B Instruct	0,8926	0,9853	0,8233	0,6863	0,9621	0,5453	0,4232	0,9979	0,3500

Kết quả cho thấy sự cải thiện ấn tượng và nhất quán trên toàn bộ năm mô hình khi được tích hợp vào hệ thống đa tác tử. Điểm nổi bật nhất là sự gia tăng vượt bậc ở chỉ số Alignment: hầu hết các mô hình đạt Alignment gần hoặc bằng 1,0 ở Cấp 2 và Cấp 3. Cụ thể, Qwen3 8B và Phi-4 đều đạt Alignment bằng 1,0000 ở Cấp 3, trong khi kết quả cơ sở tương ứng chỉ là 0,2141 và 0,2348. Điều này chứng tỏ tác tử *Phân loại* hoạt động rất hiệu quả trong việc đảm bảo câu hỏi sinh ra đúng cấp độ nhận thức yêu cầu.

Một phát hiện quan trọng là các mô hình có kết quả cơ sở yếu nhất lại hưởng lợi nhiều nhất từ hệ thống đa tác tử. Phi-4, với Distractor Quality ở Cấp 1 chỉ đạt

0,4860 khi hoạt động độc lập, đã tăng lên 0,9407 khi kết hợp đa tác tử (cải thiện 93,6%). Llama 3.1 8B Instruct, mô hình có kết quả cơ sở thấp nhất, cũng ghi nhận mức cải thiện Acceptance ở Cấp 3 từ 0,0200 lên 0,4232 (tăng hơn 21 lần). Tuy nhiên, ngay cả với sự hỗ trợ của đa tác tử, Distractor Quality ở Cấp 3 của Llama vẫn chỉ đạt 0,3500, cho thấy hệ thống đa tác tử có thể bù đắp nhưng không thể hoàn toàn thay thế năng lực nội tại của mô hình nền.

5.1.3 Kết quả nhóm LLM thương mại

Bảng 5.3 trình bày kết quả của ba mô hình LLM thương mại, đóng vai trò đường cơ sở để đánh giá mức độ cạnh tranh của giải pháp sLLM kết hợp đa tác tử.

Bảng 5.3: Kết quả đánh giá nhóm LLM thương mại trên bộ dữ liệu RACE-High

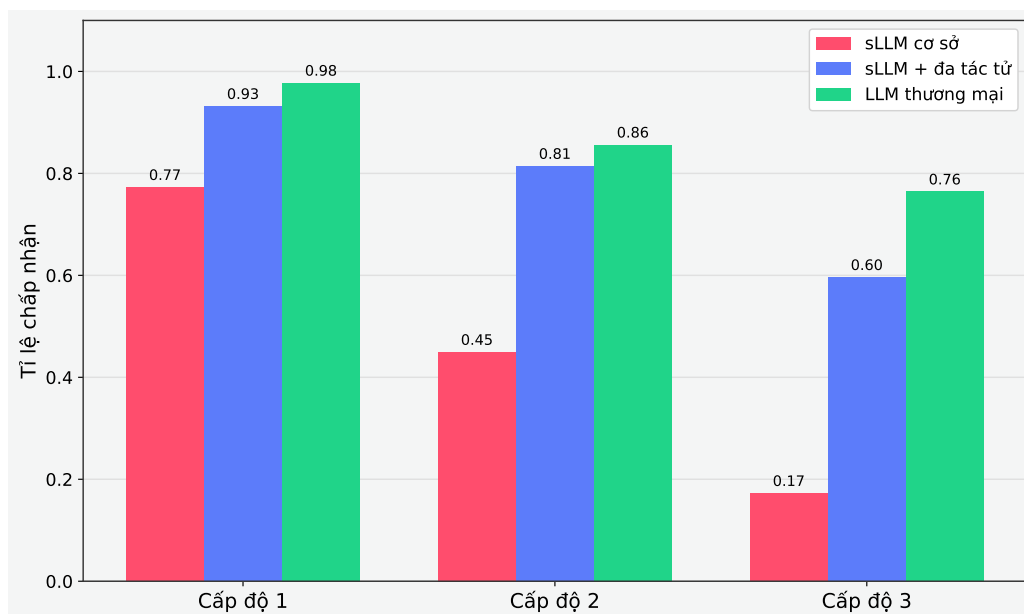
Mô hình	Cấp 1 - Nhớ			Cấp 2 - Hiểu			Cấp 3 - Phân tích		
	Acc.	Align.	Dist.	Acc.	Align.	Dist.	Acc.	Align.	Dist.
Gemini 3 Flash	0,9820	0,9900	0,9647	0,9280	0,9320	0,8980	0,7753	0,8138	0,8980
GPT-5 Mini	0,9800	0,9920	0,9733	0,8240	0,8320	0,7387	0,8448	0,9032	0,8047
Claude Haiku 4.5	0,9700	0,9900	0,9453	0,8160	0,8340	0,7273	0,6720	0,7540	0,6380

Nhóm LLM thương mại thể hiện chất lượng ổn định và đồng đều hơn so với nhóm sLLM cơ sở. Ở Cấp 1, cả ba mô hình đều đạt Acceptance trên 0,97 và Distractor Quality trên 0,94, cho thấy năng lực vượt trội trong việc sinh câu hỏi cấp thấp.

Gemini 3 Flash dẫn đầu tổng thể với Acceptance cao nhất ở Cấp 1 (0,9820) và Cấp 2 (0,9280), đồng thời đạt Distractor Quality xuất sắc ở Cấp 3 (0,8980). Đáng chú ý, GPT-5 Mini thể hiện sức mạnh đặc biệt ở Cấp 3 với Acceptance đạt 0,8448 - cao nhất trong toàn bộ nhóm thương mại, cho thấy khả năng sinh câu hỏi phân tích vượt trội. Claude Haiku 4.5, mặc dù là mô hình nhỏ nhất trong nhóm, vẫn đạt kết quả cạnh tranh với Acceptance ở Cấp 1 là 0,9700 và Distractor Quality ở Cấp 2 là 0,7273.

5.1.4 So sánh tổng hợp

Hình 5.1 trình bày biểu đồ so sánh Acceptance trung bình của ba nhóm mô hình theo từng cấp độ nhận thức, qua đó cho thấy khác biệt về năng lực.



Hình 5.1: So sánh chỉ số Acceptance trung bình giữa ba nhóm mô hình (sLLM cơ sở, sLLM kết hợp MAS, LLM thương mại) theo ba cấp độ nhận thức Bloom

Kết quả so sánh cho thấy ba phát hiện chính. Thứ nhất, ở Cấp 1 và Cấp 2, nhóm sLLM kết hợp đa tác tử đạt kết quả cạnh tranh sát sao với nhóm LLM thương mại. Cụ thể, Phi-4 kết hợp đa tác tử đạt Acceptance 0,9753 ở Cấp 1 và 0,9072 ở Cấp 2, trong khi Gemini 3 Flash đạt 0,9820 và 0,9280 tương ứng - chênh lệch không đáng kể. Thứ hai, ở Cấp 3, nhóm LLM thương mại vẫn duy trì ưu thế rõ rệt, đặc biệt ở chỉ số Distractor Quality, nơi Gemini 3 Flash đạt 0,8980 trong khi Gemma 3 12B IT kết hợp đa tác tử chỉ đạt 0,6600. Thứ ba, khoảng cách chất lượng giữa nhóm sLLM cơ sở và hai nhóm còn lại là rất lớn ở Cấp 2 và Cấp 3, khẳng định vai trò thiết yếu của kiến trúc đa tác tử hoặc năng lực mô hình lớn trong việc sinh câu hỏi tư duy bậc cao.

5.2 Kết quả OCR

Phần này đánh giá khả năng OCR của bảy mô hình: bốn mô hình thị giác-ngôn ngữ quy mô nhỏ chuyên dụng và ba mô hình ngôn ngữ lớn thương mại đa phương thức, trên hai điều kiện: ảnh sạch và ảnh nhiễu mô phỏng điều kiện thực tế. Hai chỉ số đánh giá là CER và WER; giá trị thấp hơn cho thấy OCR tốt hơn.

5.2.1 Kết quả trên ảnh sạch

Bảng 5.4 trình bày kết quả OCR trên tập ảnh sạch, nơi văn bản được chụp trong điều kiện lý tưởng với độ phân giải cao và không có nhiễu.

Bảng 5.4: Kết quả OCR trên ảnh sạch

Mô hình	CER	WER
DeepSeek OCR 2	0,0681	0,0948
DotOCR 1.5	0,0075	0,0172
MinerU 2.5	0,0252	0,0427
PaddleOCR VL 1.5	0,0113	0,0232
Gemini 3 Flash	0,0084	0,0141
GPT-5 Mini	0,0150	0,0209
Claude Haiku 4.5	0,0238	0,0337

Trên ảnh sạch, DotOCR 1.5 đạt kết quả tốt nhất trong nhóm mô hình nhỏ gọn với CER chỉ 0,0075 và WER 0,0172, tương đương với hơn 99% ký tự được nhận dạng chính xác. Gemini 3 Flash đạt kết quả gần tương đương với CER 0,0084 và WER 0,0141, là mô hình thương mại có hiệu suất OCR tốt nhất. PaddleOCR VL 1.5 xếp thứ ba với CER 0,0113. Trong nhóm thương mại, GPT-5 Mini (CER 0,0150) và Claude Haiku 4.5 (CER 0,0238) đều đạt kết quả tốt, trong khi DeepSeek OCR 2 có CER cao nhất (0,0681). Nhìn chung, trên ảnh sạch, khoảng cách giữa các mô hình nhỏ gọn tốt nhất và mô hình thương mại là không đáng kể.

5.2.2 Kết quả trên ảnh nhiễu

Bảng 5.5 trình bày kết quả OCR trên tập ảnh đã qua tăng cường dữ liệu (data augmentation), mô phỏng các điều kiện chụp ảnh thực tế như nghiêng, mờ, nhiễu hạt và thay đổi ánh sáng.

Kết quả trên ảnh nhiễu cho thấy sự phân hóa rõ rệt giữa hai nhóm mô hình. Gemini 3 Flash thể hiện khả năng kháng nhiễu vượt trội nhất: CER chỉ tăng từ 0,0084 lên 0,0204 (tăng 2,4 lần), trong khi các mô hình nhỏ gọn tốt nhất tăng từ 10 đến 20 lần. GPT-5 Mini cũng duy trì hiệu suất ổn định với CER 0,0269 trên ảnh nhiễu. Hai mô hình thương mại này vượt trội hẳn so với toàn bộ nhóm mô hình nhỏ gọn

Bảng 5.5: Kết quả OCR trên ảnh nhiễu

Mô hình	CER	WER
DeepSeek OCR 2	0,1158	0,1579
DotOCR 1.5	0,1467	0,1771
MinerU 2.5	0,1079	0,2769
PaddleOCR VL 1.5	0,1143	0,1473
Gemini 3 Flash	0,0204	0,0326
GPT-5 Mini	0,0269	0,0409
Claude Haiku 4.5	0,1799	0,1932

trong điều kiện nhiễu.

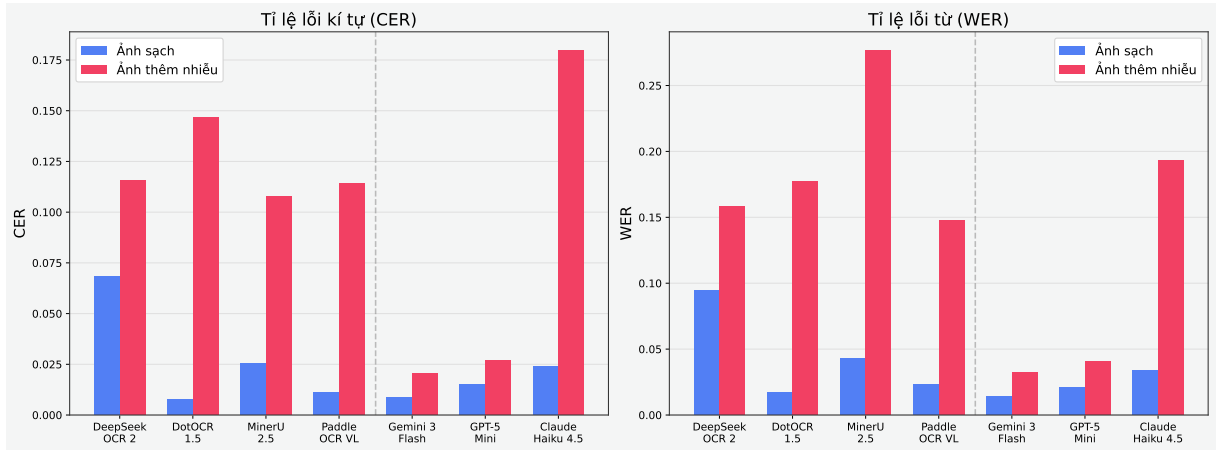
Trong nhóm mô hình nhỏ gọn, MinerU 2.5 đạt CER thấp nhất (0,1079) nhưng lại có WER cao nhất (0,2769), cho thấy mô hình này nhận dạng ký tự tương đối tốt nhưng gặp khó khăn trong việc ghép từ chính xác. PaddleOCR VL 1.5 thể hiện sự cân bằng tốt nhất với cả CER (0,1143) và WER (0,1473) đều ở mức trung bình thấp. DotOCR 1.5, mô hình tốt nhất trên ảnh sạch, lại chịu mức suy giảm lớn nhất: CER tăng gần 20 lần từ 0,0075 lên 0,1467.

Đáng chú ý, Claude Haiku 4.5 là ngoại lệ trong nhóm thương mại: CER tăng từ 0,0238 lên 0,1799 (tăng 7,6 lần), ngang mức suy giảm của các mô hình nhỏ gọn. Kết quả này cho thấy khả năng kháng nhiễu không hoàn toàn tương quan với quy mô mô hình, mà phụ thuộc vào chiến lược huấn luyện và dữ liệu tiền xử lý của từng kiến trúc. Hình 5.2 so sánh trực quan hiệu suất OCR giữa hai điều kiện ảnh, cho thấy rõ mức độ suy giảm khi chuyển từ ảnh sạch sang ảnh nhiễu.

5.3 Phân tích chi phí

Bên cạnh chất lượng, chi phí sử dụng API là một yếu tố quan trọng trong việc lựa chọn giải pháp triển khai thực tế. Bảng 5.6 trình bày chi phí trung bình cho việc xử lý 100 đoạn văn mẫu (tương đương tối đa 1.500 câu hỏi) của từng cấu hình mô hình, được theo dõi thông qua OpenRouter API.

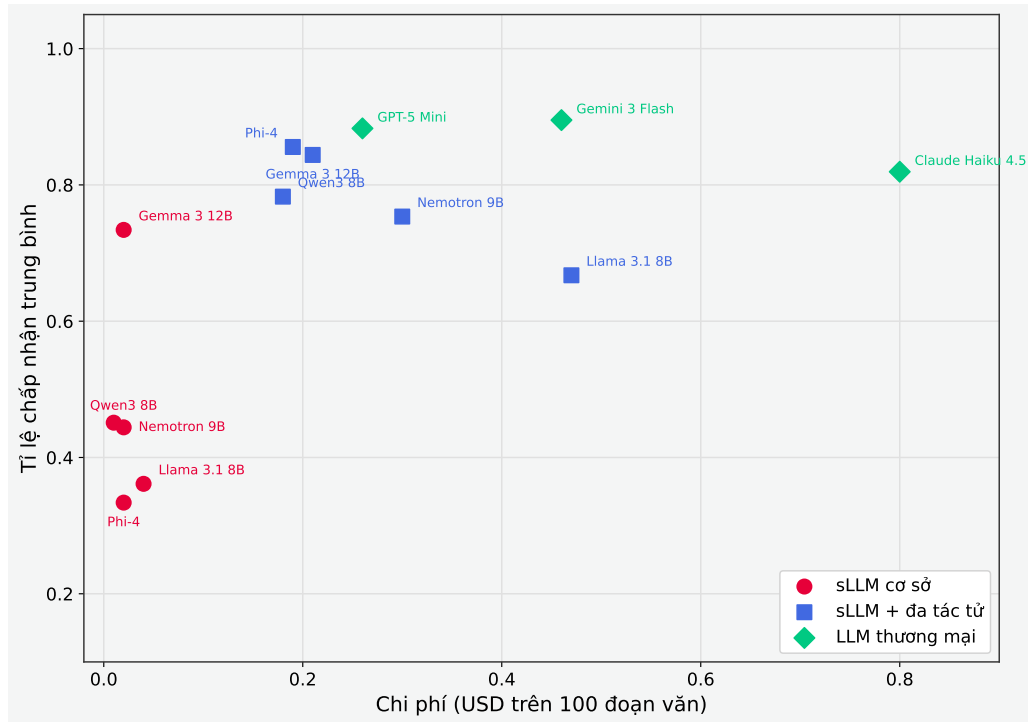
Hình 5.3 trình bày biểu đồ phân tích mối quan hệ giữa chi phí và chất lượng (đo bằng Acceptance trung bình trên ba cấp độ) cho từng cấu hình mô hình.



Hình 5.2: So sánh CER và WER giữa điều kiện ảnh sạch và ảnh nhiễu cho bảy mô hình OCR, minh họa mức độ suy giảm hiệu suất khi chuyển sang điều kiện thực tế

Bảng 5.6: Chi phí sử dụng API cho 100 đoạn văn mẫu (USD)

Nhóm	Mô hình	Chi phí (USD)
sLLM cơ sở	Gemma 3 12B IT	\$0,02
	Qwen3 8B	\$0,01
	Phi-4	\$0,02
	Nemotron Nano 9B v2	\$0,02
	Llama 3.1 8B Instruct	\$0,04
sLLM + đa tác tử	Gemma 3 12B IT	\$0,21
	Qwen3 8B	\$0,18
	Phi-4	\$0,19
	Nemotron Nano 9B v2	\$0,30
	Llama 3.1 8B Instruct	\$0,47
LLM thương mại	Gemini 3 Flash	\$0,46
	GPT-5 Mini	\$0,26
	Claude Haiku 4.5	\$0,80



Hình 5.3: Biểu đồ phân tán chi phí API (USD) so với Acceptance trung bình trên ba cấp độ nhận thức, xác định biên Pareto tối ưu giữa chất lượng và ngân sách

Phân tích chi phí cho thấy ba điểm đáng chú ý. Thứ nhất, nhóm sLLM cơ sở có chi phí cực thấp (từ \$0,01 đến \$0,04 cho 100 đoạn văn), tuy nhiên chất lượng ở Cấp 2 và Cấp 3 không đáp ứng được yêu cầu thực tế. Thứ hai, hệ thống đa tác tử làm tăng chi phí khoảng 10 đến 20 lần so với sLLM cơ sở (ví dụ: Gemma từ \$0,02 lên \$0,21, Qwen3 từ \$0,01 lên \$0,18), nhưng mức cải thiện chất lượng thu được là vượt trội - đặc biệt ở Cấp 2 và Cấp 3 - tạo ra tỷ suất lợi ích trên chi phí rất cao. Thứ ba, so với nhóm LLM thương mại, cấu hình Gemma 3 12B IT kết hợp đa tác tử nổi lên như lựa chọn tối ưu theo biên Pareto: chỉ với \$0,21 (bằng 45,7% chi phí của Gemini 3 Flash và 26,3% chi phí của Claude Haiku 4.5), cấu hình này đạt chất lượng cạnh tranh ở Cấp 1, Cấp 2 và duy trì kết quả chấp nhận được ở Cấp 3.

5.4 Thảo luận

Từ kết quả thực nghiệm được trình bày ở các phần trên, nghiên cứu rút ra sáu nhận định chính.

Hệ thống đa tác tử giúp tăng chất lượng cho sLLM. Kết quả cho thấy kiến trúc đa tác tử không chỉ cải thiện chất lượng mà còn thể hiện một đặc tính quan

trọng: các mô hình có kết quả cơ sở yếu nhất lại hưởng lợi nhiều nhất. Phi-4, với Distractor Quality ở Cấp 1 chỉ đạt 0,4860 khi hoạt động đơn lẻ, đã tăng lên 0,9407 khi kết hợp đa tác tử - mức cải thiện 93,6%. Llama 3.1 8B Instruct ghi nhận cải thiện Acceptance ở Cấp 3 hơn 21 lần. Điều này gợi ý rằng hệ thống đa tác tử hoạt động hiệu quả nhất khi mô hình nền có năng lực tiềm ẩn chưa được khai thác đầy đủ trong chế độ hoạt động đơn lẻ.

Tác tử *Phân loại* hoạt động hiệu quả nhờ cơ chế chống thiên lệch. Chỉ số Alignment đạt gần hoặc bằng 1,0 ở Cấp 2 và Cấp 3 cho hầu hết các mô hình trong nhóm đa tác tử (Qwen3 8B và Phi-4 đạt 1,0000 ở Cấp 3), chứng tỏ việc xáo trộn ngẫu nhiên câu hỏi trước khi phân loại giúp đảm bảo tính khách quan trong quá trình phân loại câu hỏi theo thang nhận thức Bloom.

Tác tử *Học sinh* đóng vai trò lọc câu hỏi mơ hồ. Bằng cách mô phỏng quá trình trả lời của học sinh, tác tử này giúp phát hiện các câu hỏi có đề bài thiếu rõ ràng, phương án nhiễu quá dễ loại trừ, hoặc đáp án không phân biệt rõ với các phương án sai. Cơ chế phản hồi từ tác tử *Học sinh* đến tác tử *Hiệu chỉnh* tạo thành vòng lặp cải tiến liên tục, giúp nâng cao chất lượng câu hỏi qua nhiều vòng.

Biên Pareto: Phi-4 kết hợp đa tác tử là lựa chọn giá trị tối ưu. Khi xem xét đồng thời chi phí và chất lượng, cấu hình Phi-4 với hệ thống đa tác tử nổi lên là điểm tối ưu trên biên Pareto. Với chi phí chỉ \$0,19 cho 100 đoạn văn, cấu hình này đạt Acceptance 0,9753 (Cấp 1), 0,9072 (Cấp 2) và 0,6845 (Cấp 3), vượt trội so với Claude Haiku 4.5 (\$0,80) ở Cấp 1 và Cấp 2, đồng thời cạnh tranh sát sao với Gemini 3 Flash (\$0,46) ở hai cấp độ này.

OCR: mô hình nhỏ gọn gặp khó khăn với ảnh nhiễu, mô hình thương mại phân hóa rõ rệt. Kết quả OCR cho thấy dù các mô hình nhỏ gọn hoạt động tốt trên ảnh sạch (DotOCR 1.5 đạt CER 0,0075), hiệu suất suy giảm đáng kể trên ảnh nhiễu (CER tăng lên 0,1079-0,1467). Trong khi đó, Gemini 3 Flash và GPT-5 Mini duy trì hiệu suất ổn định trên ảnh nhiễu (CER lần lượt 0,0204 và 0,0269), nhưng Claude Haiku 4.5 lại chịu mức suy giảm tương đương nhóm mô hình nhỏ gọn (CER 0,1799). Phát hiện này nhấn mạnh rằng khả năng kháng nhiễu phụ thuộc vào chiến lược huấn luyện chứ không hoàn toàn vào quy mô mô hình, đồng thời cho thấy vai trò quan trọng của bước tiền xử lý ảnh trong xử lý thực tế.

Hạn chế của phương pháp đánh giá LLM-as-a-Judge. Toàn bộ kết quả đánh giá trong nghiên cứu này được thực hiện bằng phương pháp G-Eval, sử dụng mô

hình ngôn ngữ lớn làm người đánh giá tự động. Mặc dù phương pháp này cho phép đánh giá quy mô lớn với chi phí thấp và tính nhất quán cao, nó vẫn thiếu sự đối chiếu với đánh giá của con người - tiêu chuẩn vàng trong đánh giá chất lượng giáo dục. Các nghiên cứu trước đã chỉ ra rằng LLM-as-a-Judge có thể mắc các thiên lệch như ưu tiên câu trả lời dài hơn hoặc thiên vị đầu ra của chính mình. Do đó, những kết quả định lượng trong nghiên cứu này cần phải được diễn giải với sự thận trọng nhất định.

Chương 6

Thiết kế và hiện thực ứng dụng Openlet

Chương này sẽ trình bày kiến trúc và xây dựng ứng dụng Web Openlet nhằm kiểm chứng khả năng ứng dụng vào thực tế của các nghiên cứu trên. Nội dung của chương sẽ bao gồm giới thiệu về ứng dụng, phân tích yêu cầu và trình bày các luồng hoạt động chính để đáp ứng nhu cầu của người dùng cuối.

6.1 Giới thiệu ứng dụng và công nghệ sử dụng

Chương 3 của khóa luận đã trình bày chi tiết các thành phần nghiên cứu cốt lõi của hệ thống Openlet, bao gồm quy trình nhận dạng văn bản từ ảnh, phương pháp sinh câu hỏi trắc nghiệm bằng kiến trúc đa tác tử và khung đánh giá chất lượng của mô hình. Trên cơ sở đó, ứng dụng Openlet được phát triển là một ứng dụng soạn thảo và làm bài kiểm tra trắc nghiệm trực tuyến áp dụng các kỹ thuật đã nghiên cứu trên.

Openlet hướng tới hai nhóm người dùng chính. Nhóm thứ nhất là giáo viên hoặc người biên soạn tài liệu học tập. Đây là nhóm người có nhu cầu lớn trong việc chuyển đổi các tài liệu học tập sang bài kiểm tra trắc nghiệm trực tuyến. Nhóm thứ hai là học sinh hoặc người làm bài. Những người này sẽ truy cập bài kiểm tra qua liên kết chia sẻ, thực hiện làm bài và nhận kết quả phản hồi.

Trong phạm vi của khóa luận này, Openlet được thiết kế là một chu trình nghiệp vụ khép kín gồm tiếp nhận tài liệu đầu vào, trích xuất nội dung từ tài liệu, sinh câu hỏi trắc nghiệm, tổ chức làm bài và theo dõi kết quả. Ứng dụng hỗ trợ ba định dạng tài liệu phổ biến là ảnh chụp tài liệu, tệp PDF và văn bản thuần.

Để hiện thực hóa kiến trúc hệ thống đã đề xuất thành một sản phẩm hoàn chỉnh, việc lựa chọn và kết hợp các nền tảng phát triển phù hợp là yếu tố tiên quyết. Công nghệ được sử dụng để phát triển mã nguồn và triển khai ứng dụng được chia thành ba nhóm chính tương ứng với ba lớp kỹ thuật của hệ thống gồm: Công nghệ phía giao diện người dùng, công nghệ máy chủ và công nghệ trí tuệ nhân tạo.

Công nghệ phía giao diện người dùng. Lớp giao diện của Openlet được xây

dựng bằng thư viện **Next.js** [30] kết hợp với **React** [18]. Lựa chọn công nghệ này cho phép hệ thống tổ chức giao diện thành các thành phần và trang rõ ràng, thuận tiện cho quá trình phát triển.

Công nghệ phía máy chủ. Để thuận tiện cho quá trình triển khai và vận hành ứng dụng, hệ thống Openlet sử dụng nền tảng **Firebase** [10] làm cơ sở dữ liệu và lưu trữ tệp tĩnh. Ngoài ra, để hệ thống có thể mở rộng linh hoạt, máy chủ được phát triển dưới dạng serverless thông qua tính năng Cloud Functions của Firebase. Cơ chế này giúp hệ thống có thể tách bạch mỗi bước nghiệp vụ trong quy trình thành một hàm thực thi độc lập.

Nhóm công nghệ mô hình trí tuệ nhân tạo. Hệ thống sử dụng các sLLM, VLM và LLM thương mại thông qua API của nền tảng trung gian **OpenRouter** [24]. Cách triển khai này cho phép hệ thống chuyển đổi linh hoạt giữa nhiều loại mô hình ngôn ngữ khi không có hạ tầng tại chỗ.

6.2 Phân tích đặc tả yêu cầu

Trong phần này, khóa luận sẽ tập trung vào các yêu cầu chức năng và phi chức năng mà nền tảng Openlet tập trung phát triển. Trong khi các yêu cầu chức năng tập trung vào các chức năng cốt lõi mà hệ thống phải có để người dùng cuối có thể sử dụng thì yêu cầu phi chức năng tập trung hơn vào trải nghiệm của người dùng và khả năng mở rộng.

6.2.1 Yêu cầu chức năng

Hệ thống Openlet được thiết kế và xây dựng xoay quanh hai tác nhân chính là *Giáo viên* và *Học sinh*. Các yêu cầu chức năng được chia làm hai nhóm tương ứng với hai tác nhân này; Hình 6.1 biểu diễn các ca sử dụng của mỗi tác nhân.

Giáo viên (Người quản trị bài kiểm tra). Đây là tác nhân có nhiệm vụ khởi tạo và quản lý toàn bộ quy trình và vòng đời của bài kiểm tra. Các chức năng cốt lõi cần có bao gồm:

- *Tạo và quản lý bài kiểm tra:* Tải lên tài liệu đầu vào; lựa chọn cấu hình AI.
- *Biên tập nội dung:* Rà soát, chỉnh sửa trực tiếp nội dung câu hỏi, phương án

trả lời và lời giải thích do hệ thống sinh ra.

- *Cấu hình và công bố*: Thiết lập cấu hình của bài kiểm tra (cho phép ẩn danh, làm lại, thời gian làm bài) và mức độ phản hồi kết quả.
- *Theo dõi và thống kê*: Quản lý danh sách các lượt nộp bài của học sinh và xem các chỉ số thống kê tổng hợp (phân bố điểm, bảng xếp hạng thành tích).

Học sinh (Người làm bài). Tác nhân này dùng đường dẫn từ giáo viên để làm bài kiểm tra trực tuyến. Các chức năng chính là:

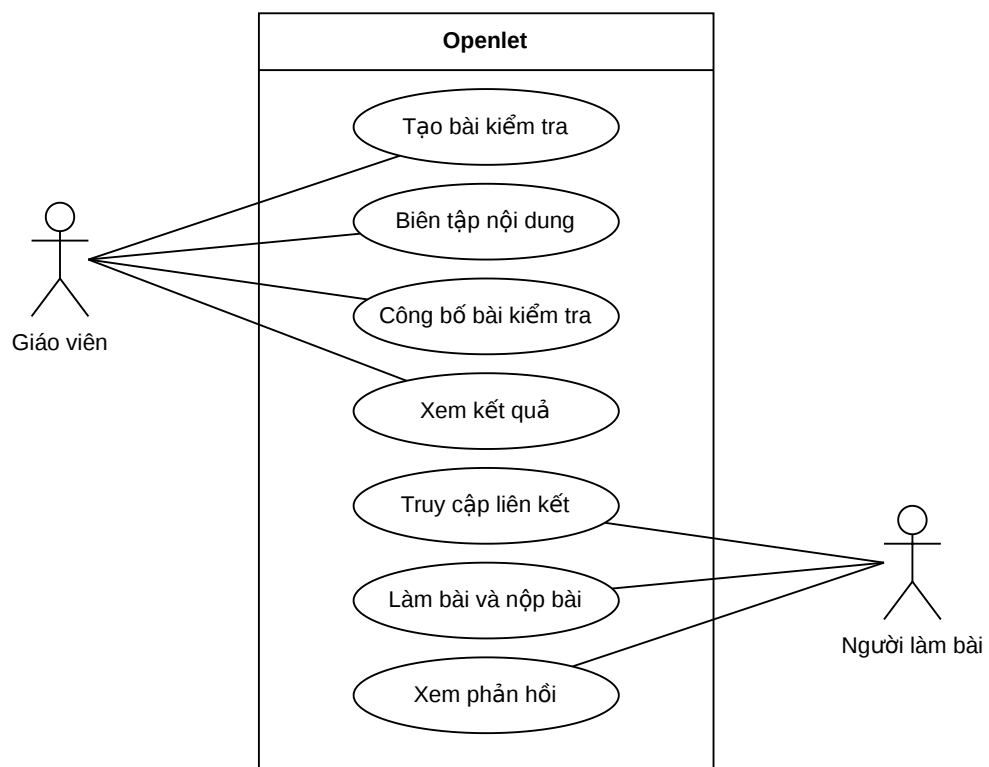
- *Truy cập và xác thực*: Truy cập bài kiểm tra qua liên kết chia sẻ; định danh hoặc làm bài ẩn danh tùy cấu hình.
- *Thực hiện bài kiểm tra*: Đọc đề thi đã được lọc đáp án, chọn phương án trả lời và nộp bài.
- *Nhận phản hồi*: Xem kết quả đánh giá, điểm số và lời giải thích chi tiết theo đúng giới hạn hiển thị của giáo viên.

6.2.2 Yêu cầu phi chức năng

Các yêu cầu phi chức năng quyết định trực tiếp cách kiến trúc hệ thống được thiết kế và triển khai có thể đáp ứng được các yêu cầu này hay không. Do vậy, việc xem xét các yêu cầu phi chức năng của nền tảng cần được triển khai đóng một vai trò quan trọng. Các yêu cầu phi chức năng dưới đây là các yêu cầu có thể được kiểm thử trực tiếp và sẽ được kiểm thử trong quá trình xây dựng và phát triển hệ thống.

Tương thích trên nhiều loại thiết bị. Hệ thống Openlet được xây dựng với triết lý đa nền tảng từ đầu. Giao diện được thiết kế có thể thay đổi linh hoạt dựa vào thiết bị và kích thước màn hình, điều này giúp giáo viên có thể dễ dàng dùng điện thoại chụp hình tài liệu từ dạng vật lý trong khi học sinh có thể sử dụng máy tính để thuận tiện cho quá trình làm bài.

Tính mở rộng và linh hoạt. Máy chủ của hệ thống phải có khả năng hỗ trợ số lượng người dùng đột biến, tự động mở rộng năng lực tính toán của máy chủ. Điều này xuất phát từ đặc điểm cốt lõi của các ứng dụng giáo dục là phải xử lý lượng lớn người dùng cùng lúc khi bắt đầu một bài kiểm tra.



Hình 6.1: Biểu đồ ca sử dụng tổng quát của Openlet với hai tác nhân chính: giáo viên (quản lý vòng đời bài kiểm tra) và người làm bài (truy cập và nộp bài)

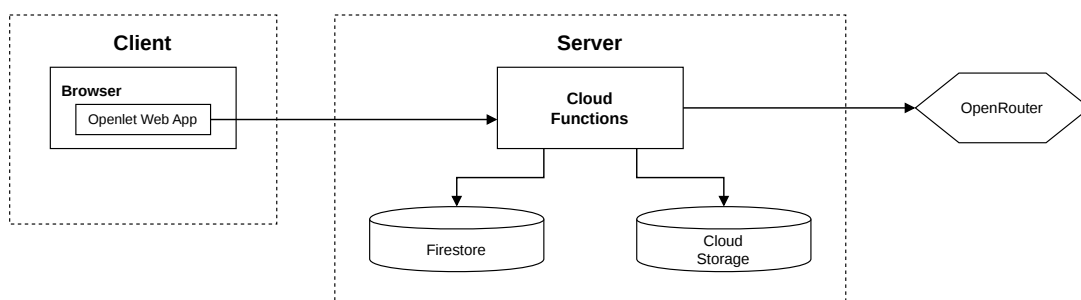
Hiệu năng và xử lý bất đồng bộ. Các tác vụ nặng phụ thuộc vào mô hình trí tuệ nhân tạo như OCR và LLM phải được xử lý dưới nền một cách bất đồng bộ mà không gây ảnh hưởng đến các chức năng khác của hệ thống. Bên cạnh đó, giao diện của người dùng phải được cập nhật trạng thái của tác vụ theo thời gian thực để đảm bảo trải nghiệm người dùng.

6.3 Thiết kế hệ thống

6.3.1 Kiến trúc tổng thể

Quá trình lựa chọn và thiết kế kiến trúc hệ thống đóng vai trò quan trọng trong việc xác định xem một hệ thống có đáp ứng được các yêu cầu về cả chức năng và phi chức năng hay không. Việc áp dụng một kiến trúc cụ thể không chỉ cần đảm bảo sự thích hợp với yêu cầu hiện tại của hệ thống mà còn phải đề cao khả năng mở rộng và phát triển trong tương lai.

Hệ thống Openlet được triển khai theo kiến trúc ba tầng. Tầng thứ nhất là giao diện Web được cài đặt bằng thư viện **Next.js** và sử dụng Firebase Authentication để xác thực tài khoản thông qua Google. Tầng tiếp theo là các khối Cloud Functions đóng vai trò xử lý từng công việc nghiệp vụ như chấm điểm, truy xuất đề thi, OCR. Tầng thứ ba là dịch vụ trung gian OpenRouter cung cấp API của các mô hình trí tuệ nhân tạo. Kiến trúc của hệ thống được biểu diễn trong Hình 6.2.



Hình 6.2: Kiến trúc ba tầng của Openlet: tầng giao diện (Next.js/React), tầng dịch vụ (Firebase), và tầng AI (OpenRouter kết nối các mô hình OCR/LLM)

Các Cloud Functions cốt lõi của hệ thống được tách theo từng bước nghiệp vụ quan trọng nhằm đảm bảo khả năng xử lý bất đồng bộ, bảo vệ dữ liệu nhạy cảm và hỗ trợ mở rộng độc lập theo tải thực tế. Bảng 6.1 mô tả vai trò của các hàm chính trong luồng vận hành của Openlet.

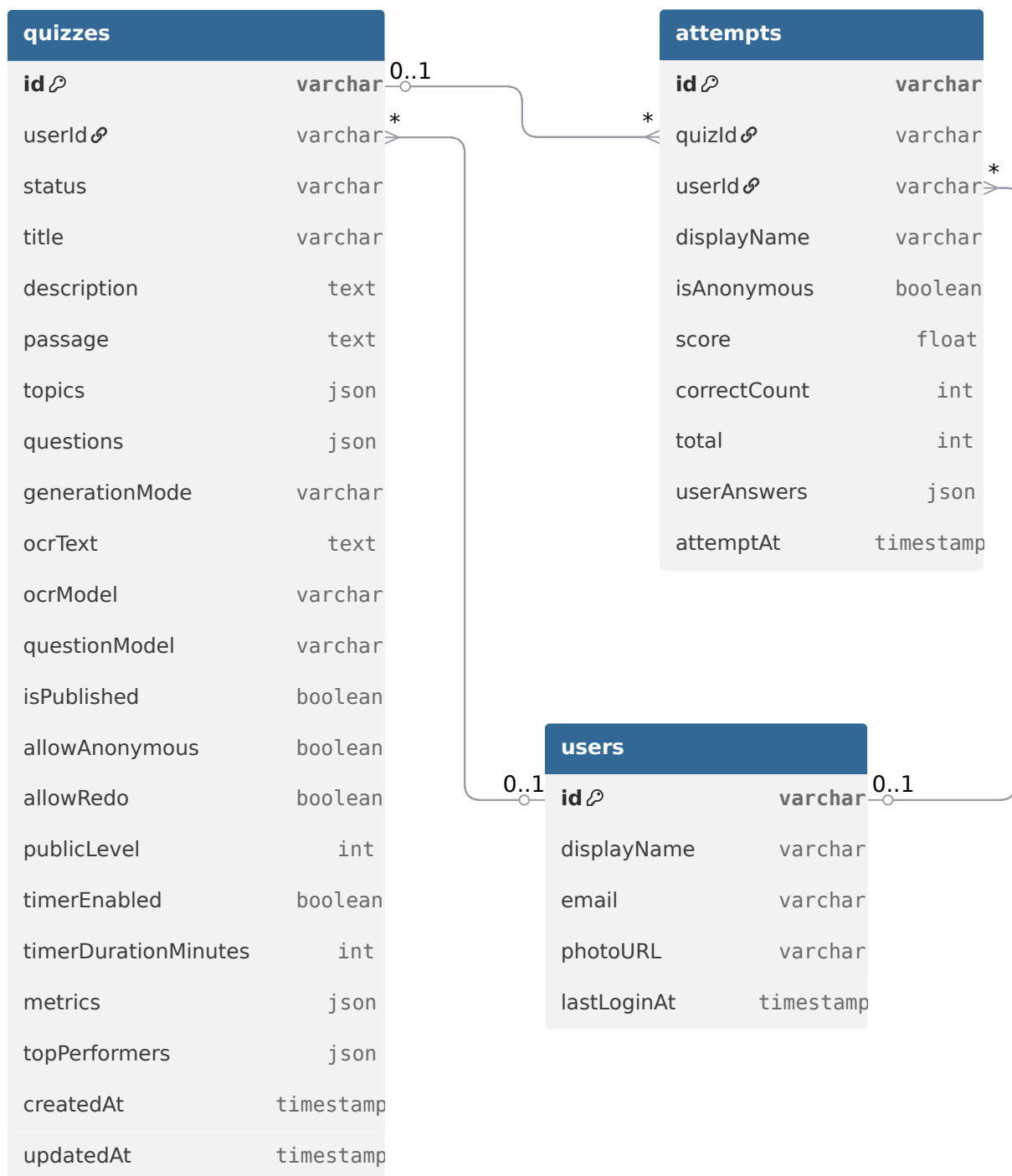
Cloud Function	Mô tả
<code>processDocument</code>	Nhận ảnh hoặc PDF từ Cloud Storage, gọi mô hình OCR và cập nhật văn bản trích xuất vào Firestore.
<code>generateQuiz</code>	Nhận văn bản đầu vào, cấu hình mô hình và chế độ sinh; gọi hệ thống sinh câu hỏi trắc nghiệm và lưu kết quả vào bản ghi bài kiểm tra.
<code>getPublicQuiz</code>	Trả về phiên bản bài kiểm tra đã lọc đáp án đúng và giải thích để người làm bài có thể truy cập an toàn qua liên kết công khai.
<code>submitAttempt</code>	Nhận đáp án của người làm bài, chấm điểm phía máy chủ, lưu lượt nộp bài và trả về phản hồi theo cấu hình của giáo viên.
<code>updateQuizStats</code>	Cập nhật các thống kê tổng hợp như số lượt làm, điểm trung bình, phân bố điểm và bảng xếp hạng thành tích.

Bảng 6.1: Các Cloud Functions cốt lõi của Openlet và vai trò trong hệ thống

6.3.2 Mô hình dữ liệu

Từ những yêu cầu chức năng cơ bản đã được phân tích trong phần 6.2, khóa luận đã cân nhắc và thiết kế lược đồ các thực thể và mối quan hệ giữa chúng được thể hiện trong Hình 6.3. Các thực thể được quản lý trong cơ sở dữ liệu bởi dịch vụ tương ứng sẽ được ký hiệu thông qua tiền tố: *users* - lưu trữ thông tin người dùng, *quizzes* - lưu trữ dữ liệu bài kiểm tra và *attempts* - lưu trữ kết quả của các lần làm bài của học sinh.

Cloud Firestore được lựa chọn để lưu trữ vì bản chất dữ liệu của hệ thống Openlet có tính chất lồng nhau rõ rệt. Cụ thể, một bài kiểm tra không chỉ có các thông tin mô tả đơn giản mà còn chứa danh sách các câu hỏi, mỗi câu hỏi lại chứa nhiều lựa chọn khác nhau. Với đặc điểm dữ liệu như này, nếu sử dụng mô hình cơ sở dữ liệu quan hệ, hệ thống sẽ cần thực hiện nhiều lần kết hợp bảng thay vì chỉ cần một lần truy vấn như Firestore. Bên cạnh đó, nền tảng còn hỗ trợ đồng bộ dữ liệu theo thời gian thực, rất phù hợp đối với những ứng dụng yêu cầu theo dõi tiến trình xử lý như Openlet.



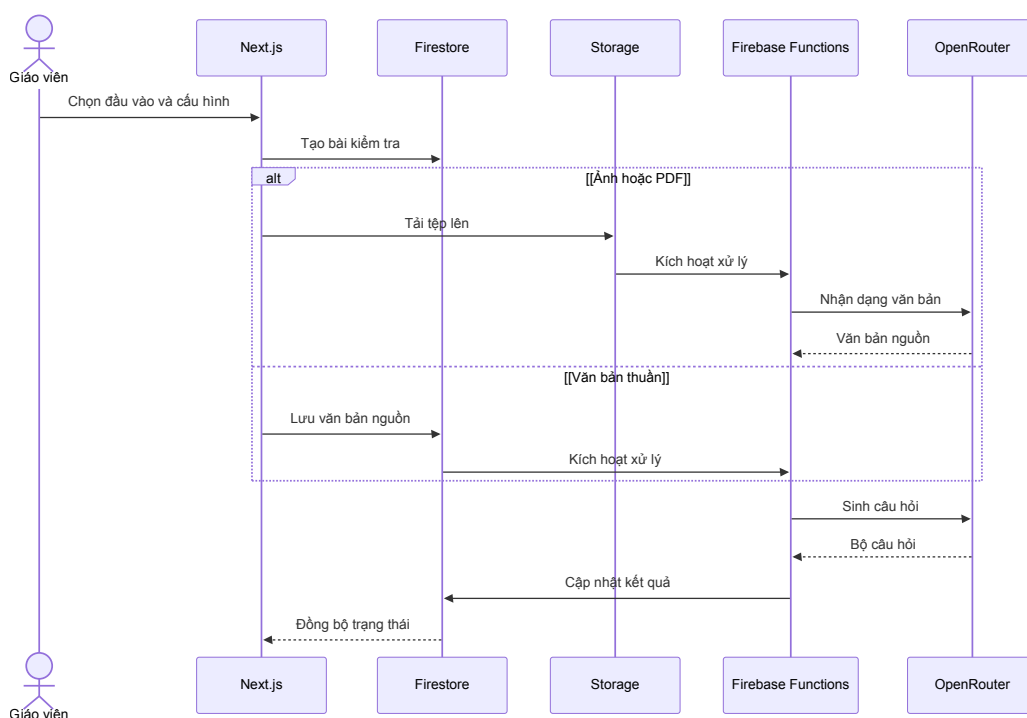
Hình 6.3: Mô hình dữ liệu chính của Openlet gồm ba bảng: **users** (hồ sơ người dùng), **quizzes** (bài kiểm tra và câu hỏi), và **attempts** (lượt làm bài và kết quả)

6.4 Hiện thực các luồng chức năng cốt lõi

Phần 6.2 đã phân tích và đưa ra các yêu cầu chức năng mà nền tảng làm bài kiểm tra trắc nghiệm trực tuyến hướng đến. Từ những thành phần hệ thống trong phần 6.3, khóa luận sẽ trình bày hiện thực hóa hai luồng chức năng chính tương ứng với hai tác nhân *Giáo viên* và *Học sinh*.

6.4.1 Luồng tạo bài kiểm tra tự động

Sơ đồ tuần tự cho chức năng tạo bài kiểm tra tự động được trình bày ở Hình 6.4 để minh họa cách giao diện, nền tảng lưu trữ dữ liệu, nền tảng lưu trữ tệp, các hàm xử lý phía máy chủ và lớp mô hình trí tuệ nhân tạo phối hợp với nhau.

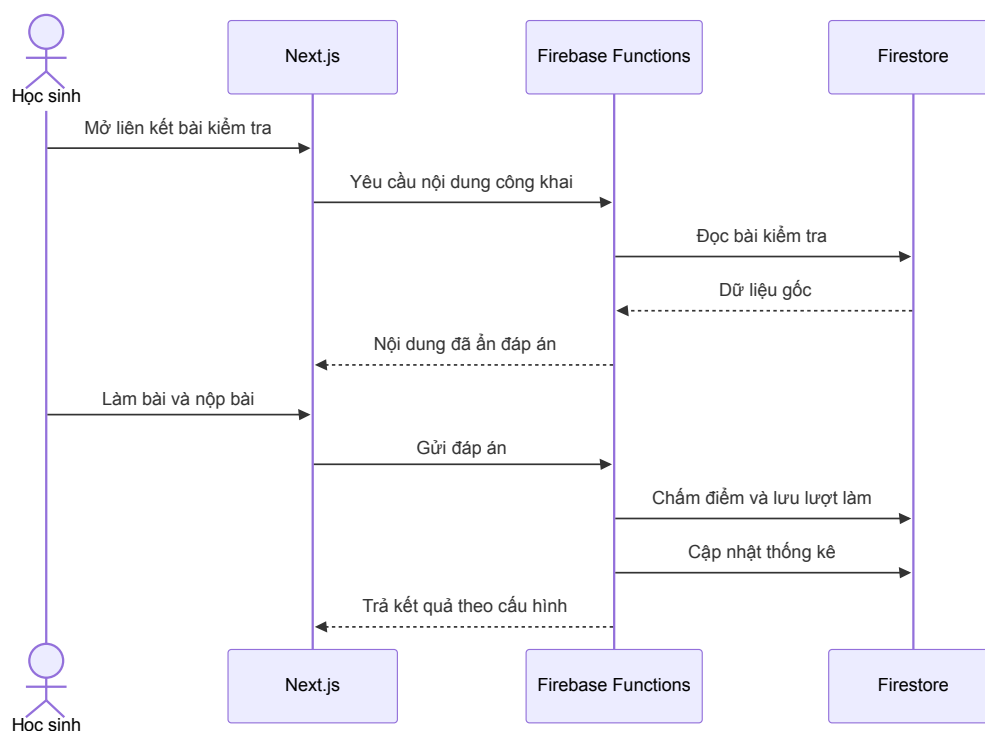


Hình 6.4: Biểu đồ tuần tự của luồng tạo bài kiểm tra tự động: từ tiếp nhận tài liệu đầu vào, qua OCR và MAS sinh câu hỏi, đến cập nhật trạng thái theo thời gian thực

Sau khi giáo viên tải tệp tài liệu văn bản lên, hệ thống sẽ khởi tạo một bản ghi để theo dõi tiến trình xử lý. Các Cloud Function sẽ được kích hoạt tuần tự: thực hiện OCR đối với ảnh hoặc tệp PDF, phân tích nội dung văn bản, sinh bài kiểm tra trắc nghiệm và cập nhật dữ liệu vào Firestore.

6.4.2 Luồng truy cập, làm bài và nộp bài

Sơ đồ tuần tự cho chức năng truy cập bài kiểm tra, làm bài và nộp bài được trình bày ở Hình 6.5 để làm rõ cách hệ thống bảo vệ đáp án đúng trong khi vẫn hỗ trợ chia sẻ công khai qua liên kết.



Hình 6.5: Biểu đồ tuần tự của luồng truy cập, làm bài và nộp bài: người làm bài nhận đề đã lọc đáp án, chấm điểm phía máy chủ, và nhận phản hồi theo mức cấu hình

Học sinh tham gia bài kiểm tra bằng cách truy cập liên kết mà giáo viên chia sẻ. Sau đó, hệ thống yêu cầu người làm bài nhập thông tin cá nhân hoặc đăng nhập với Google. Khi người dùng nộp bài, máy chủ đảm nhiệm việc chấm điểm, lưu lịch sử bài làm vào cơ sở dữ liệu. Cuối cùng hệ thống trả về điểm số kèm theo đáp án nếu giáo viên cấu hình cho phép.

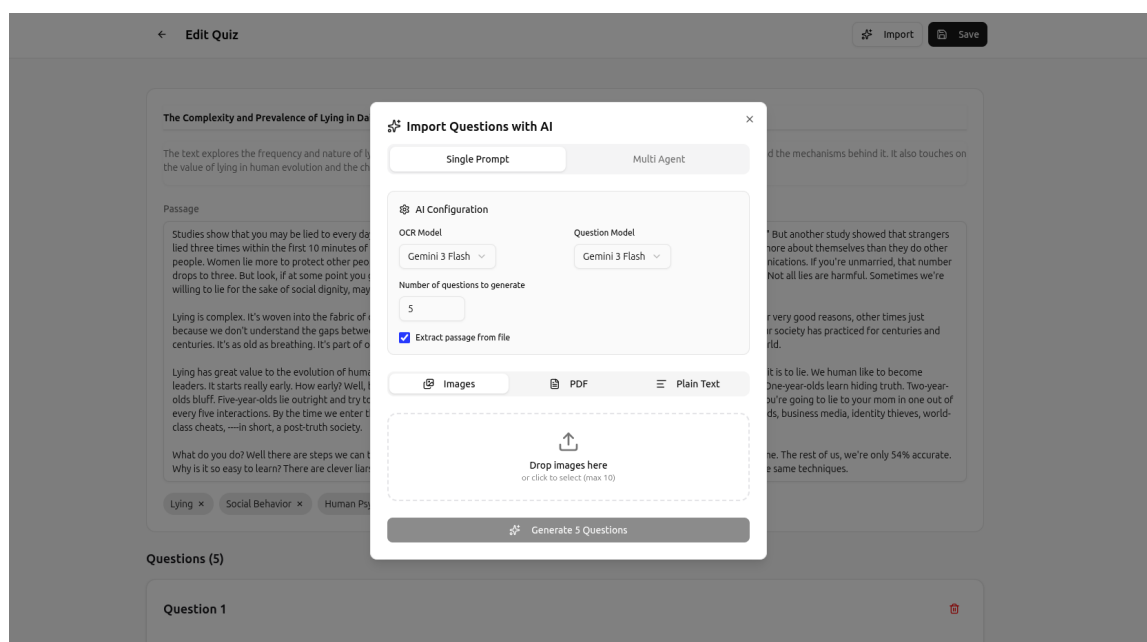
6.5 Giao diện tiêu biểu của ứng dụng

6.5.1 Giao diện dành cho giáo viên

Giao diện quản trị của *Giáo viên* được tổ chức liền mạch từ trang danh sách bài kiểm tra đến trang biên soạn bài kiểm tra. Trong môi trường biên soạn, giáo viên

có thể chỉnh sửa câu hỏi, đáp án đúng hoặc tải tập tài liệu để hệ thống chuyển thành các câu hỏi trắc nghiệm bằng kiến trúc đa tác tử đề xuất.

Hình 6.6 minh họa màn hình biên tập trung tâm của giáo viên. Giáo viên nhìn thấy ngay kết quả sinh tự động, có thể sửa từng câu hỏi tại chỗ và không phải chuyển qua nhiều màn hình trung gian. Cách tổ chức này phù hợp với mục tiêu của ứng dụng là hỗ trợ giáo viên hoàn thiện nhanh một bài kiểm tra trước khi công bố.

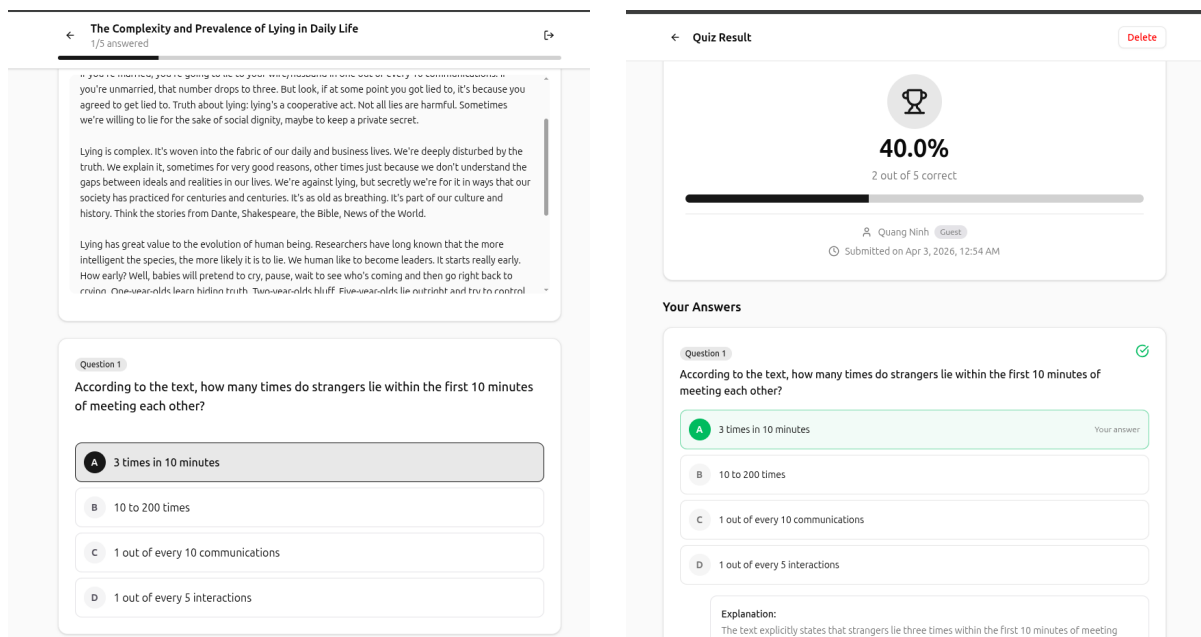


Hình 6.6: Giao diện biên tập bài kiểm tra dành cho giáo viên: hiển thị câu hỏi đã sinh tự động với các công cụ chỉnh sửa nội dung, phương án và cấp độ nhận thức

6.5.2 Giao diện dành cho người làm bài

Giao diện dành cho người làm bài được tối giản hơn đáng kể so với giao diện của giáo viên. Người học truy cập từ liên kết công khai, nhập tên hiển thị hoặc đăng nhập bằng tài khoản, sau đó được điều hướng tới màn hình làm bài. Trong quá trình làm bài, giao diện hiển thị rõ nội dung câu hỏi, các phương án, tiến độ làm bài và bộ đếm thời gian khi giáo viên bật tính năng này.

Hình 6.7 cho thấy cách Openlet tổ chức trải nghiệm của người làm bài theo hướng liền mạch. Màn hình làm bài ưu tiên khả năng tập trung và thao tác nhanh, trong khi màn hình kết quả ưu tiên tính rõ ràng của thông tin phản hồi. Sự phân tách này giúp người dùng không bị quá tải thông tin trong lúc làm bài, đồng thời vẫn nhận được mức hỗ trợ phù hợp sau khi hoàn thành.



Hình 6.7: Giao diện làm bài (trái) và trang kết quả (phải): người làm bài chọn đáp án với bộ đếm thời gian, sau đó nhận phản hồi theo mức hiển thị cấu hình bởi giáo viên

Nhìn chung, lớp giao diện của Openlet phản ánh đúng triết lý thiết kế của toàn hệ thống: giáo viên được cung cấp một không gian biên soạn và kiểm soát linh hoạt, còn người làm bài tiếp cận một trải nghiệm gọn, rõ và an toàn. Đây là điều kiện cần để các kết quả nghiên cứu ở Chương 3 có thể được đưa vào sử dụng trong một ngữ cảnh nghiệp vụ thực tế.

Chương 7

Kết luận và Hướng phát triển

7.1 Tổng kết

Nghiên cứu này đã đề xuất và phát triển thành công một hệ thống tự động sinh câu hỏi trắc nghiệm đa cấp độ nhận thức từ dữ liệu ảnh tài liệu, giải quyết hai bài toán cốt lõi trong quy trình số hóa giáo dục: (1) OCR chính xác từ ảnh chụp chất lượng kém và (2) sinh MCQ theo thang nhận thức Bloom với độ khó và chất lượng sư phạm được kiểm soát chặt chẽ. Đóng góp nổi bật nhất của nghiên cứu là việc thiết kế một kiến trúc MAS sáng tạo, kết hợp sLLM với các cơ chế tự kiểm duyệt (tác tử Học sinh, tác tử Phân loại, và tác tử Hiệu chỉnh). Kết quả thực nghiệm cho thấy giải pháp này không chỉ khắc phục được hạn chế của sLLM khi hoạt động độc lập mà còn đạt được hiệu năng ngang ngửa với các LLM thương mại lớn ở cấp độ câu hỏi Nhớ và Hiểu, mở ra tiềm năng ứng dụng mạnh mẽ, tối ưu chi phí cho các môi trường tài nguyên hạn chế.

7.2 Hạn chế

Mặc dù đạt được những kết quả khả quan, nghiên cứu vẫn tồn tại một số hạn chế nhất định. Thứ nhất, hệ thống vẫn gặp khó khăn trong việc sinh các câu hỏi ở cấp độ Phân tích (Cấp 3), ngay cả khi có sự hỗ trợ của MAS, do giới hạn về năng lực suy luận nội tại của các sLLM hiện tại. Thứ hai, thực nghiệm hiện tại mới chỉ được tiến hành trên một tập dữ liệu tiếng Anh duy nhất (RACE), chưa được kiểm chứng đầy đủ trên các miền tri thức khác (như toán học, khoa học tự nhiên) hoặc các ngôn ngữ khác, đặc biệt là tiếng Việt. Thứ ba, phương pháp đánh giá hoàn toàn dựa trên cơ chế đánh giá tự động bằng mô hình (LLM-as-a-Judge), thiếu sự đối chiếu độc lập với đánh giá từ các chuyên gia sư phạm thực tế, điều này có thể tiềm ẩn các sai lệch chưa được nhận diện đầy đủ.

7.3 Hướng phát triển

Dựa trên các kết quả và hạn chế đã phân tích, nghiên cứu định hướng một số hướng phát triển cốt lõi trong tương lai:

- **Mở rộng đa ngôn ngữ và tiếng Việt:** Tích hợp và đánh giá hệ thống trên dữ liệu tiếng Việt, xây dựng các bộ lọc và tác tử chuyên biệt để xử lý đặc thù ngữ pháp và ngữ nghĩa, hướng tới triển khai thực tiễn trong nước.
- **Đánh giá bởi chuyên gia:** Tổ chức các đợt đánh giá độc lập với sự tham gia của giáo viên và chuyên gia giáo dục để thu thập phản hồi thực tế, từ đó tinh chỉnh bộ tiêu chí đánh giá tự động và cải thiện tính sư phạm của các phương án nhiều.
- **Tinh chỉnh mô hình:** Sử dụng tập dữ liệu MCQ chất lượng cao do MAS sinh ra để tinh chỉnh trực tiếp các sLLM. Hướng đi này kỳ vọng sẽ giúp sLLM nội tại hóa khả năng sinh câu hỏi khó, giảm sự phụ thuộc vào kiến trúc MAS tồn kém trong giai đoạn triển khai.
- **Tích hợp hệ thống học tập thích ứng:** Đưa mô-đun sinh câu hỏi vào một hệ thống quản lý học tập, cho phép tự động điều chỉnh độ khó của câu hỏi tiếp theo dựa trên năng lực và phản hồi tức thời của người học.

Tài liệu tham khảo

- [1] Marah Abdin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, Russell J Hewett, Mojan Javaheripi, Piero Kauffmann, et al. Phi-4 technical report. *arXiv preprint arXiv:2412.08905*, 2024.
- [2] Lorin W Anderson and David R Krathwohl. *A taxonomy for learning, teaching, and assessing: A revision of Bloom’s taxonomy of educational objectives: complete edition*. Addison Wesley Longman, Inc., 2001.
- [3] Anthropic. Introducing claude haiku 4.5. <https://www.anthropic.com/news/claude-haiku-4-5>, 2025.
- [4] Benjamin S Bloom, Max D Engelhart, Edward J Furst, Walker H Hill, David R Krathwohl, et al. *Taxonomy of educational objectives: The classification of educational goals. Handbook 1: Cognitive domain*. Longman New York, 1956.
- [5] Harrison Chase. Langchain. <https://github.com/langchain-ai/langchain>, 2022.
- [6] Cheng Cui, Ting Sun, Suyin Liang, Tingquan Gao, Zelun Zhang, Jiaxuan Liu, Xueqing Wang, Changda Zhou, Hongen Liu, Manhui Lin, Yue Zhang, Yubo Zhang, Handong Zheng, Jing Zhang, Jun Zhang, Yi Liu, Dianhai Yu, and Yanjun Ma. Paddleocr-vl: Boosting multilingual document parsing via a 0.9b ultra-compact vision-language model, 2025.
- [7] Sabina Elkins, Ekaterina Kochmar, Iulian Serban, and Jackie CK Cheung. How useful are educational questions generated by large language models? In *International Conference on Artificial Intelligence in Education*, pages 536–542. Springer, 2023.
- [8] Cosimo Galeone, Minsu Park, Giuseppe Ettorre, and Daniele Ligorio. When correct isn’t usable: Improving structured output reliability in small language models. *arXiv preprint arXiv:2605.02363*, 2026.
- [9] Google. Google fonts. <https://fonts.google.com/>, 2010.
- [10] Google. Firebase. <https://firebase.google.com/>, 2011.

- [11] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [12] Alexander Groleau, Kok Wei Chee, Stefan Larson, Samay Maini, and Jonathan Boarman. Augraphy: A data augmentation library for document images. 2022.
- [13] Michael Heilman and Noah A Smith. Good question! statistical ranking for question generation. In *Human language technologies: The 2010 annual conference of the North American Chapter of the Association for Computational Linguistics*, pages 609–617, 2010.
- [14] Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard Hovy. Race: Large-scale reading comprehension dataset from examinations. In *Proceedings of the 2017 conference on empirical methods in natural language processing*, pages 785–794, 2017.
- [15] LangChain Team. LangGraph: Build stateful, multi-actor applications with LLMs. <https://langchain-ai.github.io/langgraph/>, 2024. Documentation and framework reference.
- [16] Yumeng Li, Guang Yang, Hao Liu, Bowen Wang, and Colin Zhang. dots.ocr: Multilingual document layout parsing in a single vision-language model, 2025.
- [17] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-eval: Nlg evaluation using gpt-4 with better human alignment. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 2511–2522, 2023.
- [18] Meta Platforms, Inc. React - a javascript library for building user interfaces. <https://react.dev/>, 2013.
- [19] Ruslan Mitkov, Ha Le An, and Nikiforos Karamanis. A computer-aided environment for generating multiple-choice test items. *Natural language engineering*, 12(2):177–194, 2006.
- [20] Hiroyuki Miura. Pyppeteer: Headless chrome/chromium automation library (unofficial port of puppeteer). <https://github.com/pyppeteer/pyppeteer>, 2018.

- [21] Junbo Niu, Zheng Liu, Zhuangcheng Gu, Bin Wang, Linke Ouyang, Zhiyuan Zhao, Tao Chu, Tianyao He, Fan Wu, Qintong Zhang, Zhenjiang Jin, et al. Mineru2.5: A decoupled vision-language model for efficient high-resolution document parsing, 2025.
- [22] Novita AI. Novita ai. <https://novita.ai/>, 2023.
- [23] NVIDIA. Nvidia nemotron nano 2: An accurate and efficient hybrid mamba-transformer reasoning model, 2025.
- [24] OpenRouter. Openrouter. <https://openrouter.ai/>, 2023.
- [25] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 conference on empirical methods in natural language processing*, pages 2383–2392, 2016.
- [26] Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*, 2025.
- [27] Ray Smith. An overview of the tesseract ocr engine. In *Ninth international conference on document analysis and recognition (ICDAR 2007)*, volume 2, pages 629–633. IEEE, 2007.
- [28] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.
- [29] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, Gaël Liu, Francesco Visin, Kathleen Kenealy, Lucas Beyer, Xiaohai Zhai, Anton Tsitsulin, Robert Busa-Fekete, Alex Feng, Noveen Sachdeva, Benjamin Coleman, Yi Gao, Basil Mustafa, Iain Barr, Emilio Parisotto, David Tian, Matan Eyal, Colin Cherry, Jan-Thorsten Peter, Danila Sinopalnikov, Surya Bhupatiraju, Rishabh Agarwal, Mehran Kazemi, Dan Malkin, Ravin

Kumar, David Vilar, Idan Brusilovsky, Jiaming Luo, Andreas Steiner, Abe Friesen, Abhanshu Sharma, Abheesht Sharma, Adi Mayrav Gilady, Adrian Goedeckemeyer, Alaa Saade, Alex Feng, Alexander Kolesnikov, Alexei Bendebury, Alvin Abdagic, Amit Vadi, András György, André Susano Pinto, Anil Das, Ankur Bapna, Antoine Miech, Antoine Yang, Antonia Paterson, Ashish Shenoy, Ayan Chakrabarti, Bilal Piot, Bo Wu, Bobak Shahriari, Bryce Petrini, Charlie Chen, Charline Le Lan, Christopher A. Choquette-Choo, CJ Carey, Cormac Brick, Daniel Deutsch, Danielle Eisenbud, Dee Cattle, Derek Cheng, Dimitris Paparas, Divyashree Shivakumar Sreepathihalli, Doug Reid, Dustin Tran, Dustin Zelle, Eric Noland, Erwin Huizenga, Eugene Kharitonov, Frederick Liu, Gagik Amirkhanyan, Glenn Cameron, Hadi Hashemi, Hanna Klimczak-Plucińska, Harman Singh, Harsh Mehta, Harshal Tushar Lehri, Hussein Hazimeh, Ian Ballantyne, Idan Szpektor, Ivan Nardini, Jean Pouget-Abadie, Jetha Chan, Joe Stanton, John Wieting, Jonathan Lai, Jordi Orbay, Joseph Fernandez, Josh Newlan, Ju yeong Ji, Jyotinder Singh, Kat Black, Kathy Yu, Kevin Hui, Kiran Vodrahalli, Klaus Greff, Linhai Qiu, Marcella Valentine, Marina Coelho, Marvin Ritter, Matt Hoffman, Matthew Watson, Mayank Chaturvedi, Michael Moynihan, Min Ma, Nabila Babar, Natasha Noy, Nathan Byrd, Nick Roy, Nikola Momchev, Nilay Chauhan, Naveen Sachdeva, Oskar Bunyan, Pankil Botarda, Paul Caron, Paul Kishan Rubenstein, Phil Culliton, Philipp Schmid, Pier Giuseppe Sessa, Pingmei Xu, Piotr Stanczyk, Pouya Tafti, Rakesh Shivanna, Renjie Wu, Renke Pan, Reza Rokni, Rob Willoughby, Rohith Vallu, Ryan Mullins, Sammy Jerome, Sara Smoot, Sertan Girgin, Shariq Iqbal, Shashir Reddy, Shruti Sheth, Siim Põder, Sijal Bhatnagar, Sindhu Raghuram Panyam, Sivan Eiger, Susan Zhang, Tianqi Liu, Trevor Yacovone, Tyler Liechty, Uday Kalra, Utku Evci, Vedant Misra, Vincent Roseberry, Vlad Feinberg, Vlad Kolesnikov, Woohyun Han, Woosuk Kwon, Xi Chen, Yinlam Chow, Yuvein Zhu, Zichuan Wei, Zoltan Egyed, Victor Cotruta, Minh Giang, Phoebe Kirk, Anand Rao, Kat Black, Nabila Babar, Jessica Lo, Erica Moreira, Luiz Gustavo Martins, Omar Sanseviero, Lucas Gonzalez, Zach Gleicher, Tris Warkentin, Vahab Mirrokni, Evan Senter, Eli Collins, Joelle Barral, Zoubin Ghahramani, Raia Hadsell, Yossi Matias, D. Sculley, Slav Petrov, Noah Fiedel, Noam Shazeer, Oriol Vinyals, Jeff Dean, Demis Hassabis, Koray Kavukcuoglu, Clement Farabet,

Elena Buchatskaya, Jean-Baptiste Alayrac, Rohan Anil, Dmitry Lepikhin, Sebastian Borgeaud, Olivier Bachem, Armand Joulin, Alek Andreev, Cassidy Hardin, Robert Dadashi, and Léonard Hussenot. Gemma 3 technical report, 2025.

- [30] Vercel. Next.js by vercel - the react framework. <https://nextjs.org/>, 2016.
- [31] Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, Yunbo Cao, Lingpeng Kong, Qi Liu, Tianyu Liu, et al. Large language models are not fair evaluators. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9440–9450, 2024.
- [32] Haoran Wei, Yaofeng Sun, and Yukun Li. Deepseek-ocr 2: Visual causal flow. *arXiv preprint arXiv:2601.20552*, 2026.
- [33] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [34] Weijie Xu, Shixian Cui, Xi Fang, Chi Xue, Stephanie Eckman, and Chandan K Reddy. Sata-bench: Select all that apply benchmark for multiple choice questions. *arXiv preprint arXiv:2506.00643*, 2025.
- [35] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [36] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.